🐍 Python Interview Preparation Handbook

From Core Concepts to Advanced System Design

Author

Urvish Bhatt

Senior Software Engineer

🔗 **GitHub:** https://github.com/Urvish-10

Programming Language *Python*

Edition *2026 — Professional Edition*

Level Coverage *Beginner → Advanced → Senior Engineer*

📘 What This Book Covers

- Core Python fundamentals
- Advanced Python internals & runtime behavior
- Object-oriented design & best practices
- Concurrency, multithreading & async programming
- Performance analysis & optimization techniques
- Python security principles & common pitfalls
- System design & distributed systems concepts
- Real-world interview pitfalls & senior-level questions

🎯 Who This Book Is For

- Software Engineers
- Backend Developers
- Data Engineers
- Python Developers
- Interview Candidates *(4–8+ years of experience)*

© 2026 Urvish Bhatt

All Rights Reserved

📘 Table of Contents

Python Interview Preparation

Topic 1: Core Python Basics

Topic 2: Control Flow

Topic 3: Built-in Data Structures

Topic 4: Comprehensions & Generator Expressions

Topic 5: Functions

Topic 6: Modules & Packaging

Topic 7: File & Data Handling

Topic 8: Error Handling & Exceptions

Topic 9: Object-Oriented Programming (OOP)

Topic 10: Advanced OOP & Internals

Topic 11: Iterators & Generators

Topic 12: Decorators & Context Managers

Topic 13: Regular Expressions (Regex)

Topic 14: Date & Time

Topic 15: Standard Library Essentials

Topic 16: Command-Line & Environment

Topic 17: Logging & Debugging

Topic 18: Typing & Code Quality

Topic 19: Concurrency & Parallelism

Topic 20: Python Runtime & Memory

Topic 21: Metaprogramming & Reflection

Topic 22: Performance & Optimization

Topic 23: Security (Conceptual)

Topic 24: System Design & Distributed Systems

Topic 25: Design Principles & Design Patterns

Topic 26: Common Python Interview Questions & Pitfalls

Topic 27: Python Behavior & Internals (Frequently Asked but Tricky)

🧩 TOPIC 1 — Core Python Basics

What is Python? Is Python interpreted or compiled?

Answer

Python is a **high-level, dynamically typed programming language**. It is **both compiled and interpreted**.

When you run a Python program:

1. The source code (`.py`) is **compiled into bytecode** (`.pyc`)
2. The bytecode is executed by the **Python Virtual Machine (PVM)**

So Python is **not**:

- A pure interpreter like Bash
- A native compiler like C/C++

It uses a **hybrid execution model**.

How it works internally

```
hello.py → bytecode (.pyc) → Python Virtual Machine → Execution
```

- `.pyc` files are **platform-independent**
- Stored inside `__pycache__/`
- Automatically reused to speed up future runs

Example

```
print("Hello")
```

When executed:

- Python compiles it into bytecode
- The PVM interprets and executes that bytecode

Common Trap

? If Python is compiled, why is it slow?

Answer: Python compiles to **bytecode**, not machine code. Bytecode still runs inside the PVM, adding overhead compared to C/C++ which compile directly to CPU instructions.

What are Python variables?

Answer

A variable in Python is **not a memory box**. It is a **name (reference) bound to an object**.

Python uses **name binding**, not direct variable storage.

Example

```
x = 10
y = x
```

Memory representation:

```
10  ← object
↑
x , y
```

Both `x` and `y` refer to the **same object**.

Important Note

- Immutable objects (int, str, tuple) cannot be modified in place
- Mutable objects (list, dict, set) can be modified via references

Common Trap

? Does `y = x` copy the value?

Answer: No. Python copies the **reference**, not the object itself.

What are Python data types?

Python provides several built-in data types:

```
Category     Types
Numbers      int, float, complex
Text         str
Boolean      bool
Null         NoneType
Containers   list, tuple, set, dict
```

Example

```
a = 10
b = 3.5
c = "hi"
d = True
e = None
```

```
print(type(a), type(c))
```

Common Trap

? Is `bool` a separate type?

Answer: `bool` is a **subclass of** `int`.

```
isinstance(True, int)  # True
```

That's why:

```
True + True == 2
```

What is dynamic typing?

Answer

Python is **dynamically typed** — **Types belong to objects, not variables**.

A variable can reference different types at different times.

Example

```
x = 10
x = "hello"
x = [1, 2, 3]
```

No error occurs — `x` simply points to different objects.

Common Trap

? Does Python perform type checking?

Answer: Yes — **at runtime**, not at compile time.

What is type conversion?

Answer

Type conversion means converting one data type into another.

There are two kinds:

- **Explicit conversion**
- **Implicit conversion**

Explicit conversion example

```
x = "10"
y = int(x)
print(y + 5)  # 15
```

Implicit conversion example

```
5 + 2.5  # 7.5
```

Python automatically converts `int` → `float`.

Common Trap

? Why does `int("10.5")` fail?

Answer: Because `"10.5"` is not a valid integer string. Convert to `float` first:

```
int(float("10.5"))
```

What is input and output?

Answer

`input()` reads data from **standard input** and **always returns a string**.

Example

```
age = input("Enter age: ")
print(type(age))  # str
```

Convert if numerical operations are required:

```
age = int(age)
```

Common Trap

? Why does `input()` return a string even for numbers?

Answer: Because input is read as **raw text** from the terminal.

What are comments and docstrings?

Comments

Used for developer notes and ignored by Python.

```
# This is a comment
```

Docstrings

Used for documentation and stored in memory.

```
def add(a, b):
    """Returns sum of two numbers"""
    return a + b

print(add.__doc__)
```

Key Difference

- Comments → ignored
- Docstrings → stored in `__doc__`

Common Trap

? Are docstrings ignored at runtime?

Answer: No. They become part of the object and can be accessed.

What is indentation?

Answer

Python uses **indentation to define code blocks**, instead of `{}`.

Example

```
if True:
    print("A")
    print("B")
```

Important Note

- Indentation is **mandatory**
- Python typically uses **4 spaces per level**

Common Trap

? Is indentation only for readability?

Answer: No. Indentation is part of Python's **syntax**. Incorrect indentation raises `IndentationError`.

🧩 TOPIC 2 — Control Flow

(if, elif, else, for, while, break, continue, pass)

What is control flow in Python?

Answer

Control flow defines **the order in which Python executes statements** in a program. It determines:

- **Which code runs**
- **How many times it runs**
- **When execution stops or skips**

Python controls execution flow using:

- Conditional statements (`if / elif / else`)
- Loops (`for`, `while`)
- Loop control keywords (`break`, `continue`, `pass`)

How does `if-elif-else` work internally?

Python evaluates conditions **from top to bottom** and executes **only the first condition that evaluates to `True`**.

Example

```
marks = 72

if marks > 90:
    print("A")
elif marks > 60:
    print("B")
else:
    print("C")
```

Output:

```
B
```

Once a condition is satisfied, Python **skips all remaining conditions**.

Important Note

- Conditions are checked **sequentially**

- Order of conditions **matters**

Common Trap

? Will Python check all conditions even after one is True?

Answer: No. Python stops checking once a matching block executes. This behavior is called **short-circuit evaluation** in condition chains.

Difference between `for` and `while`

`for` loop	`while` loop
Iterates over a sequence	Runs while a condition is `True`
Usually finite	Can become infinite
Safer and cleaner	Risky if condition isn't updated
Best for known iterations	Best for unknown iterations

Example

```
for i in range(3):
    print(i)
```

```
i = 0
while i < 3:
    print(i)
    i += 1
```

Output (both):

```
0
1
2
```

Common Trap

? Which loop is better?

Answer:

- Use `for` when you know **how many times**
- Use `while` when you don't know **when to stop**

What is `break`?

`break` **terminates the loop immediately**, skipping all remaining iterations.

Example

```
for i in range(10):
    if i == 4:
        break
    print(i)
```

Output:

```
0
1
2
3
```

Important Note

- `break` exits the **nearest enclosing loop**
- Code after the loop continues normally

Common Trap

? Does `break` exit only the `if` or the loop?

Answer: It exits the **loop**, not just the `if` block.

What is `continue`?

`continue` **skips the current iteration** and moves to the next iteration of the loop.

Example

```
for i in range(5):
    if i == 2:
        continue
    print(i)
```

Output:

```
0
1
```

```
3
4
```

Common Trap

? Does `continue` stop the loop?

Answer: No. It only skips the **current iteration**, not the entire loop.

What is `pass`?

`pass` is a **do-nothing placeholder statement**. It is used when Python expects a statement syntactically, but no action is required.

Example

```
if True:
    pass
```

This code does nothing but is **syntactically valid**.

Typical Use Cases

- Empty functions or classes
- Stub code during development
- Placeholder in conditional blocks

Common Trap

? Is `pass` the same as `continue`?

Answer: No.

`pass`	`continue`
Does nothing	Skips iteration
Execution continues normally	Jumps to next loop cycle
Can be used anywhere	Used only inside loops

What is `else` with loops?

A loop's `else` block runs **only if the loop completes normally**, meaning **no `break` was encountered**.

Example (normal completion)

```

```

```
for i in range(3):
    print(i)
else:
    print("Finished")
```

Output:

```
0
1
2
Finished
```

Example (with `break`)

```
for i in range(3):
    if i == 1:
        break
else:
    print("Finished")  # Not executed
```

Common Trap

? When does loop `else` NOT execute?

Answer: When the loop exits due to a `break` statement.

Can a loop run zero times?

Yes.

Example

```
while False:
    print("Hi")  # Never runs
```

Explanation

- Python checks the loop condition **before entering**
- If the condition is `False`, the body is skipped entirely

Common Trap

? Does Python execute a loop body at least once?

Answer: No. Python evaluates the condition **before** execution.

🧩 TOPIC 3 — Built-in Data Structures

(List, Tuple, Set, Dictionary, Mutability, Shallow vs Deep Copy)

What is a List?

Answer

A list is an **ordered, mutable, dynamic array** that can store **mixed data types**.

- Ordered → elements maintain insertion order
- Mutable → elements can be modified
- Dynamic → size can grow or shrink at runtime

Example

```
nums = [1, 2, 3]
nums.append(4)
print(nums)  # [1, 2, 3, 4]
```

Lists resize automatically when elements are added or removed.

Performance Note

- Index access → **O(1)**
- Append at end → **amortized O(1)**
- Insert/delete in middle → **O(n)**

Common Trap

? Why are lists slow for inserts in the middle?

Answer: Lists are implemented as **dynamic arrays**. Inserting in the middle requires **shifting elements**, which takes linear time.

What is a Tuple?

Answer

A tuple is an **ordered, immutable** collection.

Once created, its contents **cannot be modified**.

Example

```
t = (1, 2, 3)
# t[0] = 10  × Error
```

Key Properties

- Faster than lists (slightly)
- Can be used as dictionary keys **if all elements are immutable**
- Memory-efficient

Common Trap

? Why can tuples be dictionary keys but lists cannot?

Answer: Tuples are immutable → their **hash value never changes**. Lists are mutable → changing them would invalidate the hash.

What is a Set?

Answer

A set is an **unordered collection of unique elements**.

- Automatically removes duplicates
- Does not support indexing

Example

```
s = {1, 2, 3, 3, 4}
print(s)  # {1, 2, 3, 4}
```

Important Notes

- Sets are implemented using **hash tables**
- Membership tests are very fast → **O(1)** average

Common Trap

? Why can't sets contain lists?

Answer: Set elements must be **hashable (immutable)**. Lists are mutable → therefore not hashable.

What is a Dictionary?

Answer

A dictionary is a **hash table** that stores **key–value pairs**.

Example

```
user = {"name": "Alice", "age": 30}
print(user["name"])
```

Key Rules

- Keys must be **hashable**
- Values can be any data type
- Maintains insertion order (Python 3.7+)

Performance

- Lookup, insert, delete → **O(1)** average time

Common Trap

? Why are dictionary lookups fast?

Answer: Because dictionaries use **hash tables**, allowing direct access via hash computation.

What is Mutability?

Answer

Mutability defines whether an object can be **modified without changing its identity (`id`)**.

Mutable vs Immutable

Mutable	Immutable
list	int
dict	str
set	tuple

Example (mutable)

```
x = [1, 2]
print(id(x))
x.append(3)
print(id(x))  # same object
```

Example (immutable)

```
s = "hi"
print(id(s))
s += "!"
print(id(s))  # new object
```

Common Trap

? Why are strings immutable?

Answer: Immutability allows strings to be:

- Safely shared
- Hashable (usable as dict keys)
- Memory optimized (string interning)

What is a Shallow Copy?

A shallow copy duplicates **only the outer object**. Inner objects are **shared references**.

Example

```
import copy

a = [[1, 2], [3, 4]]
b = copy.copy(a)

b[0][0] = 99
print(a)  # [[99, 2], [3, 4]]
```

Both `a` and `b` reference the same inner lists.

Common Use

- Faster
- Safe only when inner objects are immutable

What is a Deep Copy?

A deep copy duplicates **all objects recursively**, creating a completely independent structure.

Example

```
import copy

a = [[1, 2], [3, 4]]
b = copy.deepcopy(a)
```

```
b[0][0] = 99
print(a)  # [[1, 2], [3, 4]]
```

No shared references exist between a and b.

Common Trap

? When is a shallow copy dangerous?

Answer: When the object contains **nested mutable objects**, because changes in one copy affect the other.

🧩 TOPIC 4 — Comprehensions & Generator Expressions

(List, Set, Dict Comprehensions + Generator Expressions)

What is a comprehension?

Answer

A comprehension is a **compact and expressive way to create collections** using a single expression instead of multiple lines of loop code.

Comprehensions are:

- **Cleaner** (less boilerplate)
- **Faster** than equivalent loops
- **More Pythonic**

They replace common loop + append patterns with declarative syntax.

List Comprehension

Syntax

```
[expression for item in iterable if condition]
```

- `expression` → what to produce
- `item` → variable for each element
- `iterable` → source sequence
- `if condition` → optional filter

Example

```
nums = [1, 2, 3, 4, 5]
squares = [x * x for x in nums if x % 2 == 0]
print(squares)  # [4, 16]
```

Equivalent loop

```
squares = []
for x in nums:
    if x % 2 == 0:
        squares.append(x * x)
```

Performance Note

- Runs faster due to optimized internal iteration
- Avoids repeated attribute lookups like `.append()`

Common Trap

? Why are list comprehensions faster?

Answer: They are implemented in **C internally**, reducing Python-level overhead and method calls.

Set Comprehension

Set comprehensions create **unique values automatically**.

Example

```
nums = [1, 2, 2, 3, 3, 4]
unique = {x for x in nums}
print(unique)  # {1, 2, 3, 4}
```

Key Points

- Removes duplicates
- Unordered
- Fast membership testing

Common Trap

? Why is the output order different?

Answer: Sets are **unordered** because they are implemented using hash tables.

Dictionary Comprehension

Dictionary comprehensions generate **key–value pairs** dynamically.

Syntax

```
{key_expr: value_expr for item in iterable}
```

Example

```
nums = [1, 2, 3]
squares = {x: x * x for x in nums}
```

```
print(squares)
```

Output:

```
{1: 1, 2: 4, 3: 9}
```

Use Cases

- Transforming dictionaries
- Swapping keys and values
- Filtering entries

Generator Expression

A generator expression **looks like a list comprehension**, but uses **parentheses `()` instead of brackets `[]`**.

Example

```
g = (x * x for x in range(5))
```

⚠ No computation happens immediately.

Generators are **lazy** — values are produced only when requested.

Example (on-demand evaluation)

```
g = (x * x for x in range(3))
print(next(g))  # 0
print(next(g))  # 1
print(next(g))  # 4
```

Each call to `next()` computes the next value.

Common Trap

? Why are generators memory efficient?

Answer: They generate values **one at a time**, instead of storing the entire sequence in memory.

List Comprehension vs Generator Expression

List Comprehension	Generator Expression

List Comprehension	Generator Expression
Stores all values	Lazy evaluation
Uses more memory	Uses minimal memory
Faster for small data	Better for large data
Can be reused	One-time use

Common Trap

? Why does a generator return nothing the second time?

Answer: Because generators are **exhausted after iteration**. Once consumed, they cannot be reused.

Important Note (Often Asked)

```
[x*x for x in range(5)]   # creates list immediately
(x*x for x in range(5))   # creates generator object
```

Use:

- **List comprehension** → when you need all results immediately
- **Generator expression** → when working with large or infinite data

---

🧩 TOPIC 5 — Functions

(Definitions, return, arguments, `*args`, `**kwargs`, lambda, recursion, scope, LEGB)

What is a function in Python?

Answer

A function is a **reusable block of code** that executes only when called. It can accept inputs (parameters) and return output.

Functions in Python are **first-class objects**, meaning they can be:

- Assigned to variables
- Passed as arguments
- Returned from other functions

Example

```
def add(a, b):
    return a + b

x = add
print(x(2, 3))  # 5
```

Common Trap

? Can a function exist without being called? Answer: Yes. Functions are objects created when Python loads the file, not when they are called.

Function Arguments

Python supports multiple kinds of arguments:

- **Positional**
- **Keyword**
- **Default**
- **`*args`**
- **`**kwargs`**

Example

```
def demo(a, b=10, *args, **kwargs):
    print(a, b, args, kwargs)

demo(1, 2, 3, 4, x=5)
```

Output:

```
1 2 (3, 4) {'x': 5}
```

Common Trap

? Why must `*args` come before `**kwargs`? Answer: Positional arguments must be collected before keyword arguments.

What is `*args`?

`*args` collects **extra positional arguments** into a tuple.

Example

```
def add(*nums):
    return sum(nums)

add(1, 2, 3, 4)
```

- `nums` becomes a tuple
- Can accept zero or more arguments

What is `**kwargs`?

`**kwargs` collects **extra keyword arguments** into a dictionary.

Example

```
def user(**data):
    print(data)

user(name="Alice", age=30)
```

Common Trap

? Can `*args` be empty? Answer: Yes. It simply becomes an empty tuple.

What is a lambda function?

Answer

A lambda is an **anonymous, single-expression function**.

Example

```
square = lambda x: x * x
print(square(5))
```

Key Points

- No name
- One expression only
- Automatically returns the result

Common Trap

? Why are lambdas limited to one line? Answer: They are designed for **simple expressions**, not multi-step logic.

What is recursion?

Answer

Recursion is when a function **calls itself** to solve smaller sub-problems.

Example

```
def fact(n):
    if n == 1:
        return 1
    return n * fact(n - 1)
```

Common Trap

? Why is recursion risky? Answer: Each call uses stack memory and can cause **stack overflow** if the base case is missing.

What is the LEGB Rule?

Python resolves variable names using the **LEGB order**:

```
Local → Enclosing → Global → Built-in
```

Example

```
x = 10

def outer():
    x = 20
    def inner():
        print(x)
    inner()

outer()  # 20
```

Python finds `x` in the **enclosing scope**.

Common Trap

? How do you modify an enclosing variable? Answer: Use the `nonlocal` keyword.

```
def outer():
    x = 10
    def inner():
        nonlocal x
        x = 20
```

🧩 TOPIC 6 — Modules & Packaging

(modules, packages, import system, `__name__`, virtual environments)

What is a module?

Answer

A module is a **Python file (`.py`)** containing variables, functions, or classes that can be imported and reused. Modules help with:

- Code reuse
- Better organization
- Namespace separation

Example

math_utils.py

```
def add(a, b):
    return a + b
```

```
import math_utils
print(math_utils.add(2, 3))
```

Common Trap

? Is `math` a module or a package? Answer: `math` is a **built-in C extension module**, not a package.

What is a package?

Answer

A package is a **directory that groups related modules**. It enables **hierarchical imports** and better project structure.

Traditionally, packages contain an `__init__.py` file.

Example

```
myapp/
|── __init__.py
|── auth.py
|── db.py
```

```
from myapp import auth
```

Common Trap

? Is `__init__.py` required? Answer: Not required in modern Python, but still used for initialization and clarity.

How does `import` work internally?

When Python executes `import x`, it:

1. Checks **`sys.modules`** (module cache)
2. Searches directories listed in **`sys.path`**
3. Loads the module once and caches it

Example

```
import sys
print("math" in sys.modules)

import math
print("math" in sys.modules)
```

Common Trap

? Why doesn't Python re-execute imported modules? Answer: Because imported modules are cached in `sys.modules`.

What is `__name__ == "__main__"`?

When a file is run directly:

```
__name__ == "__main__"
```

When imported:

```
__name__ == "module_name"
```

Example

```python
def main():
    print("Run directly")

if __name__ == "__main__":
    main()
```

Common Trap

? Why is this used? Answer: To prevent code from running automatically when the file is imported.

What is a virtual environment?

Answer

A virtual environment is an **isolated Python environment** with its own dependencies. It helps avoid:

- Package version conflicts
- Global site-packages pollution

Example

```
python -m venv venv
source venv/bin/activate
```

Each project manages its own packages independently.

Common Trap

? Does a virtual environment copy Python? Answer: No. It links to the Python interpreter and isolates `site-packages`.

🧩 TOPIC 7 — File & Data Handling

(File handling, JSON, serialization, deserialization, Pickle basics)

What is file handling in Python?

Answer

File handling allows Python to:

- Read data from files
- Write data to files
- Update existing files

Python provides the built-in `open()` function for file operations.

File modes

```
Mode  Meaning
r     Read
w     Write (overwrite)
a     Append
rb    Read binary
wb    Write binary
```

Example

```
f = open("data.txt", "w")
f.write("Hello")
f.close()
```

Better way (recommended)

```
with open("data.txt", "w") as f:
    f.write("Hello")
```

`with` automatically closes the file, even if an error occurs.

Common Trap

? What happens if a file is not closed? Answer: File descriptors remain open, causing **resource or memory leaks**.

Reading files

```
with open("data.txt", "r") as f:
    content = f.read()
```

Other common methods:

- `readline()` → reads one line
- `readlines()` → reads all lines into a list

Common Trap

? Difference between `read()` and `readlines()`? Answer:

- `read()` → entire content as a single string
- `readlines()` → list of lines

What is serialization?

Answer

Serialization is the process of converting a **Python object into a storable or transferable format** (such as JSON or bytes) so it can be saved or sent over a network.

```
Python object → JSON / bytes
```

What is deserialization?

Answer

Deserialization is converting serialized data **back into a Python object** for use in a program.

JSON handling

JSON supports only basic data types:

```
dict, list, string, number, boolean, null
```

Example

```
import json
```

```
data = {"name": "Alice", "age": 30}
json_str = json.dumps(data)
print(json_str)

obj = json.loads(json_str)
print(obj)
```

File-based JSON

```
with open("user.json", "w") as f:
    json.dump(data, f)

with open("user.json") as f:
    data = json.load(f)
```

Common Trap

? Why can't JSON store Python objects directly? Answer: JSON is language-independent and supports only primitive data types.

What is Pickle?

Answer

Pickle serializes **entire Python objects** into a **binary format**, preserving structure and types.

Example

```
import pickle

data = {"a": 1, "b": 2}

with open("data.pkl", "wb") as f:
    pickle.dump(data, f)

with open("data.pkl", "rb") as f:
    obj = pickle.load(f)
```

Key Difference from JSON

- Pickle → Python-specific, supports complex objects
- JSON → Language-independent, safer

Common Trap

? Is pickle safe? Answer: × No. Unpickling untrusted data can execute malicious code. **Never load pickle files from unknown sources.**

🧩 TOPIC 8 — Error Handling & Exceptions

What is an exception in Python?

Answer (detailed)

An exception is an **event raised at runtime** when Python encounters an error condition (e.g., divide by zero, file not found). Instead of silently failing, Python creates an **exception object** and stops normal execution flow. If the exception is **handled**, the program can continue; otherwise it **terminates** and prints a traceback (error report). Exceptions help separate normal logic from error-handling logic, making programs more reliable.

Example

```
x = 10 / 0  # ZeroDivisionError
```

Python stops unless handled using `try/except`.

Common Trap

? Is a SyntaxError an exception? Answer: Not in the runtime sense. `SyntaxError` happens **during parsing (before execution begins)**, while most exceptions happen **during execution**.

What is `try` and `except`?

Answer (detailed)

The `try` block contains code that **might raise an exception**. If an exception occurs, Python immediately stops executing the remaining lines inside `try` and searches for a matching `except`. The `except` block defines **how to recover** or respond (log, default value, show message, retry, etc.). If no matching `except` exists, the exception propagates upward and may crash the program.

Example

```
try:
    x = int("abc")
except ValueError:
    print("Conversion failed")
```

Common Trap

? Does Python execute remaining `try` statements after an exception? Answer: No. Once an exception is raised, the remaining statements inside `try` are skipped.

Can we catch multiple exceptions?

Answer (detailed)

Yes. You can handle multiple exceptions using:

- **Multiple `except` blocks** (more specific handling)
- A **tuple of exceptions** in one block (common handling)

Python matches exception handlers **top to bottom**, so order matters: always place **more specific** exceptions before broader ones.

Example (separate handlers)

```
try:
    x = int(input())
    y = 10 / x
except ValueError:
    print("Invalid number")
except ZeroDivisionError:
    print("Cannot divide by zero")
```

Example (grouped)

```
try:
    ...
except (ValueError, ZeroDivisionError):
    print("Invalid input")
```

Common Trap

? Which `except` block runs first? Answer: Python checks `except` blocks **top to bottom** and runs the first one that matches.

What is `else` in exception handling?

Answer (detailed)

The `else` block runs **only when the `try` block succeeds with no exception**. It's useful because it separates:

- Risky code (`try`)
- Success logic (`else`)

This keeps `try` blocks smaller and avoids accidentally catching exceptions from code that shouldn't be inside the `try`.

Example

```
try:
    x = int("10")
except ValueError:
    print("Error")
else:
    print("Conversion successful")
```

Common Trap

? Will `else` run if an exception occurs? Answer: No. If any exception occurs in `try`, `else` is skipped.

What is `finally`?

Answer (detailed)

`finally` always executes **no matter what**:

- Exception occurs or not
- `return` is executed
- `break`/`continue` happens

It is mainly used for **cleanup** such as closing files, releasing locks, and closing database/network connections. Even if an exception is raised and not caught, `finally` runs before the program exits.

Example

```
try:
    f = open("data.txt")
    print(f.read())
finally:
    f.close()
```

Common Trap

? Does `finally` execute after `return`? Answer: Yes. `finally` runs before the function actually exits.

What are custom exceptions?

Answer (detailed)

Custom exceptions are user-defined error types used to represent **application-specific failures** (business rules). They improve clarity because errors become meaningful (`AgeError`, `PaymentFailedError`) instead of generic ones. They also allow catching only the errors you care about without mixing them with unrelated system exceptions.

Example

```python
class AgeError(Exception):
    pass

def check_age(age):
    if age < 18:
        raise AgeError("Underage")

check_age(15)
```

Best Practice (commonly expected)

- Inherit from `Exception` (or a subclass)
- Use meaningful names ending with `Error`
- Optionally store extra fields (age, user_id, etc.)

Common Trap

? Why inherit from `Exception`? Answer: Python's exception system recognizes and catches errors reliably only when they inherit from `Exception`.

🧩 TOPIC 9 — Object-Oriented Programming (OOP)

(class, object, constructor, variables, methods, inheritance, polymorphism, encapsulation, abstraction, ABC, MRO)

What is Object-Oriented Programming?

Answer (detailed)

Object-Oriented Programming (OOP) is a paradigm that structures programs around **objects**, which bundle data (attributes) and behavior (methods). It models real-world entities, making code **modular, reusable, and easier to maintain**. OOP improves scalability by separating responsibilities and reducing tight coupling between components.

What is a class?

Answer (detailed)

A class is a **blueprint** used to create objects. It defines:

- **Attributes** (data)
- **Methods** (behavior)

In Python, classes themselves are objects and are created at runtime.

Example

```
class User:
    def greet(self):
        print("Hello")
```

Common Trap

? Does a class occupy memory? Answer: Yes. A class is an object and occupies memory once it is defined.

What is an object?

Answer (detailed)

An object is an **instance of a class**. It represents a concrete entity created from the class blueprint and stores actual data in memory. Each object has its own identity (`id`) and state.

Example

```
u = User()
u.greet()
```

Common Trap

? Can multiple objects share the same class? Answer: Yes. Multiple objects can be created from the same class, each with independent instance data.

What is `__init__`?

Answer (detailed)

`__init__` is a **constructor method** that runs automatically after an object is created. It initializes instance variables and prepares the object for use. It does not create the object—it only configures it.

Example

```
class User:
    def __init__(self, name):
        self.name = name
```

Common Trap

? Does `__init__` create the object? Answer: No. Object creation is handled by `__new__`; `__init__` only initializes it.

Instance variables vs Class variables

Answer (detailed)

- **Instance variables** belong to individual objects
- **Class variables** are shared across all objects of the class

Changing a class variable affects all instances unless overridden at the object level.

Example

```
class User:
    company = "ABC"    # class variable
    def __init__(self, name):
        self.name = name  # instance variable
```

Common Trap

? Why does modifying a class variable affect all objects? Answer: Because class variables live in the class namespace, shared by all instances.

Instance method, Class method, Static method

Answer (detailed)

- **Instance methods** work with object data (`self`)
- **Class methods** work with class-level data (`cls`)
- **Static methods** are utility functions related to the class

Example

```
class Demo:
    def inst(self):
        pass

    @classmethod
    def cls(cls):
        pass

    @staticmethod
    def stat():
        pass
```

Common Trap

? Why use `classmethod` instead of `staticmethod`? Answer: Because class methods receive `cls`, allowing access to and modification of class-level data.

Inheritance

Answer (detailed)

Inheritance allows a class to **reuse and extend** the behavior of another class. It supports hierarchical relationships and reduces code duplication.

Example

```
class Animal:
    def speak(self):
        print("Sound")

class Dog(Animal):
    pass
```

Common Trap

? Does Python support multiple inheritance? Answer: Yes. A class can inherit from multiple parent classes.

Polymorphism

Answer (detailed)

Polymorphism allows the **same method name** to behave differently based on the object type. Python supports polymorphism through **dynamic typing** and method overriding.

Example

```
class Dog:
    def speak(self):
        print("Bark")

class Cat:
    def speak(self):
        print("Meow")
```

Common Trap

? Does Python support method overloading? Answer: Not traditionally. Python uses default arguments and dynamic typing instead.

Encapsulation

Answer (detailed)

Encapsulation restricts direct access to internal data and exposes only what is necessary. It protects object state and reduces accidental misuse.

Example

```
class User:
    def __init__(self):
        self._age = 20       # protected
        self.__salary = 50000  # private
```

Common Trap

? Is private truly private in Python? Answer: No. Python uses name mangling (`_ClassName__var`), not strict access control.

Abstraction

Answer (detailed)

Abstraction hides implementation details and exposes only **essential behavior**. It enforces design contracts and improves consistency across implementations.

Python provides abstraction using **Abstract Base Classes (ABC)**.

Example

```
from abc import ABC, abstractmethod

class Payment(ABC):
    @abstractmethod
    def pay(self):
        pass
```

Common Trap

? Can we create an object of an abstract class? Answer: No. All abstract methods must be implemented before instantiation.

Method Resolution Order (MRO)

Answer (detailed)

MRO defines the order in which Python looks for methods in inheritance hierarchies. Python uses the **C3 linearization algorithm** to ensure a consistent and predictable lookup order, especially in multiple inheritance.

Example

```
class A: pass
class B(A): pass
class C(B): pass

print(C.mro())
```

Common Trap

? Why is MRO important? Answer: It prevents ambiguity in multiple inheritance and ensures predictable method resolution.

🧩 TOPIC 10 — Advanced OOP & Internals

(Dunder methods, dataclasses, `__slots__`, descriptors, monkey patching)

What are magic (dunder) methods?

Answer (detailed)

Magic (dunder) methods are **special methods** whose names start and end with double underscores (`__`). They define how objects behave with **built-in Python operations** like printing, arithmetic, comparison, iteration, etc. Python calls these methods **automatically**, enabling operator overloading and custom object behavior.

Example

```
class User:
    def __str__(self):
        return "User object"

u = User()
print(u)  # calls __str__
```

Common Trap

? Do we call dunder methods manually? Answer: Normally no. They are invoked implicitly by Python (e.g., `+`, `print()`, `len()`).

`__repr__` vs `__str__`

Answer (detailed)

- `__str__` → **user-friendly**, readable output
- `__repr__` → **developer-friendly**, unambiguous representation

If `__str__` is missing, Python falls back to `__repr__`.

Example

```
class User:
    def __str__(self):
        return "Alice"

    def __repr__(self):
        return "User(name='Alice')"
```

Common Trap

? Which one is used inside lists and dicts? Answer: `__repr__` is used for container display.

What are dataclasses?

Answer (detailed)

Dataclasses automatically generate **boilerplate methods** like `__init__`, `__repr__`, and `__eq__`. They are ideal for **data-holding classes** and reduce repetitive code. Introduced in Python 3.7.

Example

```
from dataclasses import dataclass

@dataclass
class User:
    name: str
    age: int
```

Common Trap

? Are dataclasses immutable by default? Answer: No. They are mutable unless `frozen=True` is specified.

What is `__slots__`?

Answer (detailed)

`__slots__` restricts objects to a **fixed set of attributes**. It prevents the creation of `__dict__` per instance, significantly reducing memory usage. Useful when creating **many lightweight objects**.

Example

```
class User:
    __slots__ = ("name", "age")

    def __init__(self, name, age):
        self.name = name
        self.age = age
```

```
u.city = "NY"  # × AttributeError
```

Common Trap

? Why does `__slots__` improve memory? Answer: It removes per-object dictionaries, which consume significant memory.

What are descriptors?

Answer (detailed)

Descriptors are objects that control **attribute access** using:

- `__get__`
- `__set__`
- `__delete__`

They form the foundation of **properties**, **ORM fields**, and many Python internals.

Example

```
class Age:
    def __get__(self, obj, objtype):
        return obj._age

    def __set__(self, obj, value):
        obj._age = value

class User:
    age = Age()
```

Common Trap

? Where are descriptors used in real projects? Answer: Frameworks like Django ORM use descriptors to manage model fields.

What is monkey patching?

Answer (detailed)

Monkey patching means **modifying existing objects or modules at runtime** without changing their source code. It is powerful but dangerous because it can cause **unexpected side effects** and debugging issues.

Example

```
def new_print():
    print("patched")

print = new_print
print()
```

Common Trap

? Is monkey patching recommended? Answer: Generally no. Use only in controlled scenarios (testing, framework internals).

🧩 TOPIC 11 — Iterators & Generators

(Iterators, generators, yield, lazy evaluation)

What is an iterator?

Answer (detailed)

An iterator is an object that returns values **one at a time** and maintains internal iteration state. Any object implementing both `__iter__()` and `__next__()` is an iterator. When exhausted, it raises `StopIteration`.

Example

```
nums = [1, 2, 3]
it = iter(nums)

print(next(it))  # 1
print(next(it))  # 2
print(next(it))  # 3
```

Common Trap

? Is a list an iterator? Answer: No. A list is an iterable; `iter(list)` returns an iterator.

What is an iterable?

Answer (detailed)

An iterable is an object that can **produce an iterator**. It does not store iteration state itself and can generate **multiple independent iterators**.

Examples: list, tuple, set, dict, string.

Example

```
nums = [1, 2, 3]
iter(nums)  # returns iterator
```

Common Trap

? Why can lists be looped multiple times but generators cannot? Answer: Iterables create new iterators; generators are iterators themselves and get exhausted.

How does a `for` loop work internally?

Answer (detailed)

A `for` loop:

1. Calls `iter()` on the iterable
2. Repeatedly calls `next()`
3. Stops when `StopIteration` is raised

This is known as the **iterator protocol**.

Conceptual example

```
it = iter(data)
while True:
    try:
        item = next(it)
    except StopIteration:
        break
```

Common Trap

? Does `for` use indexing internally? Answer: No. It uses the iterator protocol, not indexes.

What is a generator?

Answer (detailed)

A generator is a **lazy iterator** created using a function with the `yield` keyword. It pauses execution at `yield`, remembers state, and resumes from the same point on the next call.

Example

```
def count():
    yield 1
    yield 2
    yield 3

g = count()
```

```
print(next(g))  # 1
print(next(g))  # 2
```

Common Trap

? Does `yield` stop execution permanently? Answer: No. It pauses execution and resumes on the next call.

Difference between `return` and `yield`

Answer (detailed)

- `return` → ends function execution permanently
- `yield` → pauses execution and produces a value

A function containing `yield` becomes a **generator function**.

Example

```
def demo():
    yield 1
    yield 2
    return 3
```

`3` is never yielded — the generator stops.

Common Trap

? Can a function have both `return` and `yield`? Answer: Yes, but once `return` executes, the generator terminates.

What is lazy evaluation?

Answer (detailed)

Lazy evaluation means values are **computed only when needed**, not in advance. Generators implement lazy evaluation, making them memory-efficient for large or infinite data streams.

Example

```
g = (x * x for x in range(10**9))
```

No values are stored in memory upfront.

Common Trap

? When should generators be preferred over lists? Answer: For large datasets, streaming data, or one-time sequential processing.

Generator expressions vs List comprehensions

List Comprehension	Generator Expression
Eager execution	Lazy execution
High memory usage	Very low memory
Reusable	One-time use

Example

```
[x*x for x in range(5)]   # list
(x*x for x in range(5))   # generator
```

Common Trap

? Why does a generator become empty after one loop? Answer: Because generators maintain state and are exhausted after iteration.

🧩 TOPIC 12 — Decorators & Context Managers

(Function decorators, class decorators, functools, context managers, `with`)

What is a decorator in Python?

Answer (detailed)

A decorator is a **design pattern** that allows you to modify or extend the behavior of a function or class **without changing its source code**. It works by **wrapping** the original function inside another function, adding logic before or after execution. Decorators are applied **at definition time**, not at runtime invocation time.

Example

```
def my_decorator(func):
    def wrapper():
        print("Before")
        func()
        print("After")
    return wrapper

@my_decorator
def greet():
    print("Hello")

greet()
```

Output:

```
Before
Hello
After
```

Common Trap

? What actually happens with `@decorator` syntax? Answer: It is syntactic sugar for:

```
greet = my_decorator(greet)
```

This happens when the function is defined, not when it is called.

Why are decorators possible in Python?

Answer (detailed)

Decorators are possible because **functions are first-class objects** in Python. This means functions can be:

- Passed as arguments
- Returned from other functions
- Assigned to variables

Since decorators receive a function and return a function, Python fully supports this pattern.

Example

```
def outer(func):
    return func

def hello():
    print("Hi")

outer(hello)()
```

Common Trap

? Can decorators be applied to classes? Answer: Yes. Class decorators receive the class object and can modify or replace it.

Decorator with arguments

Answer (detailed)

When a decorator itself requires arguments, **three nested functions** are needed:

1. Outer function → receives decorator arguments
2. Middle function → receives the function being decorated
3. Inner function → wraps execution logic

This matches how Python applies decorators step by step.

Example

```
def repeat(n):
    def decorator(func):
        def wrapper():
            for _ in range(n):
                func()
        return wrapper
    return decorator

@repeat(3)
```

```
def hello():
    print("Hi")
```

Common Trap

? Why are three functions required? Answer: Because Python first evaluates decorator arguments, then passes the target function, and finally executes the wrapper.

What is `functools.wraps`?

Answer (detailed)

`functools.wraps` preserves the **original function's metadata**, such as:

- `__name__`
- `__doc__`
- Annotations

Without it, the decorated function appears as the wrapper, which breaks introspection, debugging, and tooling.

Example

```
from functools import wraps

def deco(func):
    @wraps(func)
    def wrapper():
        return func()
    return wrapper
```

Common Trap

? Why is losing metadata dangerous? Answer: It breaks logging, debugging, documentation generators, and frameworks that rely on function metadata.

What is a context manager?

Answer (detailed)

A context manager **automates resource management**, ensuring setup and cleanup happen correctly. It guarantees that cleanup code runs **even if an exception occurs**. Commonly used for files, locks, sockets, and database connections.

Example

```

```

```
with open("data.txt") as f:
    content = f.read()
```

The file is closed automatically after the block.

Common Trap

? What problem do context managers solve? Answer: They prevent **resource leaks** by guaranteeing cleanup under all circumstances.

How does the `with` statement work internally?

Answer (detailed)

The `with` statement:

1. Calls `__enter__()` at the start
2. Executes the block
3. Calls `__exit__()` at the end

`__exit__()` runs **even if an exception occurs**.

Example

```
class File:
    def __enter__(self):
        print("Enter")
    def __exit__(self, exc_type, exc_val, exc_tb):
        print("Exit")

with File():
    print("Working")
```

Output:

```
Enter
Working
Exit
```

Common Trap

? Can `__exit__` suppress exceptions? Answer: Yes. If `__exit__()` returns `True`, the exception is suppressed.

`contextlib` module

Answer (detailed)

`contextlib` allows creating context managers using **generator syntax**, avoiding full class definitions. It is cleaner and ideal for **simple setup/teardown logic**.

Example

```
from contextlib import contextmanager

@contextmanager
def my_context():
    print("Start")
    yield
    print("End")
```

```
with my_context():
    print("Inside")
```

Common Trap

? Why use `contextlib` instead of class-based managers? Answer: For simpler cases — it's shorter, clearer, and easier to maintain.

🧩 TOPIC 13 — Regular Expressions (Regex)

(Patterns, `match`, `search`, `findall`, groups, validation)

What is a regular expression?

Answer (detailed)

A regular expression (regex) is a **pattern-matching language** used to search, extract, replace, or validate text. It allows complex string processing using concise symbolic patterns. In Python, regex support is provided by the built-in **`re` module**.

Regex is especially useful for:

- Validation (email, phone, IDs)
- Parsing structured text
- Searching and extracting patterns

Example

```
import re

text = "My number is 9876543210"
re.search(r"\d+", text)
```

This finds the first sequence of digits.

Common Trap

? Is regex faster than normal string methods? Answer: No. Regex is more powerful but usually **slower** than simple methods like `in`, `startswith()`, or `split()`.

What are regex patterns?

Answer (detailed)

Regex patterns are **strings that describe matching rules** for characters. They use special symbols (metacharacters) to represent character types, repetition, and position in text.

Patterns are compiled internally into a state machine for matching.

Common patterns

Pattern	Meaning
`\d`	digit

Pattern	Meaning
`\w`	word character (letter, digit, _)
`.`	any character
`*`	zero or more

Pattern	Meaning
`+`	one or more
`^`	start of string
`$`	end of string

Example

```
re.search(r"\d{4}", "Year 2024")
```

Match: `2024`

Common Trap

? Why use raw strings (`r""`) for regex? Answer: Python also treats `\` as an escape character. Raw strings prevent double escaping and make regex clearer and safer.

Difference between `match`, `search`, and `findall`

Answer (detailed)

- `match()` → checks **only at the start** of the string
- `search()` → scans the **entire string**
- `findall()` → returns **all matching substrings**

Example

```
import re

text = "abc 123 xyz 456"

re.match(r"\d+", text)     # None
re.search(r"\d+", text)    # '123'
re.findall(r"\d+", text)  # ['123', '456']
```

Common Trap

? Why does `match()` return None even if digits exist? Answer: Because `match()` only checks from the **beginning of the string**.

What are groups in regex?

Answer (detailed)

Groups allow capturing **specific parts of a match** using parentheses `()`. Captured groups can be accessed later for extraction, rearranging, or validation. They are essential when working with structured data.

Example

```
import re

text = "email: test@gmail.com"
match = re.search(r"(\w+)@(\w+)\.(\w+)", text)

print(match.group(1))  # test
print(match.group(2))  # gmail
```

- `group(0)` → full match
- `group(1..n)` → captured subgroups

Common Trap

? Difference between capturing and non-capturing groups? Answer:

- Capturing: `(pattern)` → stores matched data
- Non-capturing: `(?:pattern)` → groups logic only, no storage

Regex for validation

Answer (detailed)

Regex is widely used to **validate string formats**, such as emails, phone numbers, or IDs. It ensures that input **matches a required structure**, but not real-world correctness.

Example

```
import re

email = "user@test.com"
if re.fullmatch(r"\w+@\w+\.\w+", email):
    print("Valid")
```

- `fullmatch()` ensures the **entire string** matches the pattern

- Preferred over `search()` for validation

Common Trap

? Should regex be used for full email validation ?

Answer: No. Regex can validate format, but cannot verify whether an email actually exists or is reachable.

🧩 TOPIC 14 — Date & Time

(Date handling, time handling, timezone basics)

How does Python handle date and time?

Answer (detailed)

Python handles date and time using the `datetime` **module**. It provides classes to create, manipulate, compare, and format date–time values. Date–time objects can be:

- **Naive** → no timezone information
- **Aware** → includes timezone and UTC offset

Correct timezone handling is essential for distributed and global systems.

Example

```
from datetime import datetime

now = datetime.now()
print(now)
```

Common Trap

? Is `datetime.now()` timezone-aware? Answer: No. By default, it returns a **naive** datetime without timezone information.

Difference between `date`, `time`, and `datetime`

Answer (detailed)

- `date` → represents **only calendar date** (year, month, day)
- `time` → represents **only time of day** (hour, minute, second)
- `datetime` → combines both date and time

Using the smallest required type keeps code clearer and simpler.

Example

```
from datetime import date, time, datetime

d = date.today()
t = time(10, 30)
dt = datetime.now()
```

Common Trap

? Why not always use `datetime`? Answer: Sometimes only date or time is needed; using `datetime` unnecessarily adds complexity.

What is `timedelta`?

Answer (detailed)

`timedelta` represents a **duration or time difference**. It is used for date arithmetic like adding days, hours, or calculating expiry times. Internally, it stores time as a combination of days, seconds, and microseconds.

Example

```
from datetime import datetime, timedelta

future = datetime.now() + timedelta(days=5)
```

Common Trap

? Can `timedelta` represent months or years? Answer: No. Months and years vary in length, so only fixed units are supported.

How do you format date and time?

Answer (detailed)

Python uses `strftime()` to convert datetime objects into formatted strings. Format codes control how each component appears (year, month, hour, etc.).

Example

```
from datetime import datetime

now = datetime.now()
print(now.strftime("%Y-%m-%d %H:%M:%S"))
```

Common Trap

? Difference between `strftime` and `strptime`? Answer:

- `strftime` → datetime ➜ string
- `strptime` → string ➜ datetime

What are timezones?

Answer (detailed)

Timezones represent regions with a specific **offset from UTC**. They are essential for accurate timestamps across different geographical locations. Modern Python uses the `zoneinfo` module (Python 3.9+) for timezone handling.

Example

```
from datetime import datetime
from zoneinfo import ZoneInfo

dt = datetime.now(ZoneInfo("Asia/Kolkata"))
print(dt)
```

Common Trap

? Why are timezones difficult to handle? Answer: Due to daylight saving changes, regional rules, and historical timezone shifts.

Naive vs Aware datetime

Answer (detailed)

- **Naive datetime** → no timezone information
- **Aware datetime** → includes timezone and UTC offset

Python prevents mixing them to avoid ambiguous comparisons.

Example

```
from datetime import datetime
from zoneinfo import ZoneInfo

naive = datetime.now()
aware = datetime.now(ZoneInfo("UTC"))
```

Common Trap

? Can we compare naive and aware datetimes?

Answer: No. Python raises a `TypeError` because the comparison would be ambiguous.

🧩 TOPIC 15 — Standard Library Essentials

(os, sys, pathlib, collections, itertools)

What is the Python Standard Library?

Answer (detailed)

The Python Standard Library is a collection of **built-in modules** that ship with Python. It provides ready-made solutions for common tasks like file handling, OS interaction, data structures, and iteration. Using it reduces external dependencies and improves portability and reliability.

Common Trap

? Why is Python called "batteries included"? Answer: Because most common programming needs are already handled by the standard library itself.

`os` module

Answer (detailed)

The `os` module provides a **procedural interface** to interact with the operating system. It is commonly used for:

- File and directory operations
- Environment variables
- Process-related tasks

Example

```
import os

print(os.getcwd())
os.mkdir("test")
```

Common Trap

? Difference between `os` and `pathlib`? Answer: `os` is string-based and procedural, while `pathlib` offers an object-oriented, cleaner API.

`sys` module

Answer (detailed)

The `sys` module provides access to **Python runtime internals**. It is used for interacting with the interpreter, command-line arguments, and module loading paths.

Example

```
import sys

print(sys.argv)   # command-line arguments
print(sys.path)   # module search paths
```

Common Trap

? What is `sys.modules`? Answer: A cache of all imported modules, preventing modules from being loaded multiple times.

`pathlib` module

Answer (detailed)

`pathlib` provides an **object-oriented abstraction** for filesystem paths. It improves readability, safety, and cross-platform compatibility compared to `os.path`.

Example

```
from pathlib import Path

p = Path("data.txt")
print(p.exists())
```

Common Trap

? Why prefer `pathlib` over `os.path`? Answer: It is cleaner, safer, cross-platform, and easier to compose complex paths.

`collections` module

Answer (detailed)

The `collections` module provides **specialized container data types** that extend Python's built-ins. They help write clearer, more efficient, and more expressive code for common patterns.

Common classes

```

```

```
defaultdict
Counter
deque
namedtuple
```

Example

```
from collections import Counter

c = Counter("banana")
print(c)
```

Common Trap

? Why use `defaultdict`? Answer: It avoids manual key existence checks and simplifies counting or grouping logic.

`itertools` module

Answer (detailed)

The `itertools` module provides **fast, memory-efficient iterator tools** for advanced looping and combinatorics. It works lazily, producing values only when needed.

Example

```
import itertools

for x in itertools.permutations([1, 2, 3], 2):
    print(x)
```

Common Trap

? Why is `itertools` memory efficient?

Answer: Because it returns iterators and uses **lazy evaluation** instead of storing all results in memory.

🧩 TOPIC 16 — Command-Line & Environment

(sys.argv, argparse, environment variables)

What are command-line arguments?

Answer (detailed)

Command-line arguments are values passed to a Python program **when it is executed**. They allow programs to accept dynamic input without changing source code. This is widely used for scripts, automation, configuration, and deployment workflows.

Example

```
python app.py hello 10
```

Inside Python:

```
import sys
print(sys.argv)
```

Output:

```
['app.py', 'hello', '10']
```

All arguments are received as **strings**.

Common Trap

? Why is `sys.argv[0]` the script name? Answer: Python automatically inserts the program name as the first argument for reference and debugging.

Limitations of `sys.argv`

Answer (detailed)

`sys.argv` provides only a **raw list of strings**. It does not support:

- Type validation
- Named arguments (`--port`)
- Default values
- Help messages

This makes it unsuitable for complex or user-friendly CLI tools.

Common Trap

? Can `sys.argv` parse named arguments like `--port`? Answer: No. Parsing and validation must be implemented manually.

What is `argparse`?

Answer (detailed)

`argparse` is a standard library module used to build **robust command-line interfaces**. It supports:

- Named and positional arguments
- Type checking
- Default values
- Auto-generated help and error messages

It is the recommended tool for production-grade CLI applications.

Example

```
import argparse

parser = argparse.ArgumentParser()
parser.add_argument("--port", type=int, default=8000)
args = parser.parse_args()

print(args.port)
```

Common Trap

? Why prefer `argparse` over `sys.argv`? Answer: Because `argparse` handles parsing, validation, error handling, and documentation automatically.

What are environment variables?

Answer (detailed)

Environment variables are **key–value pairs stored outside application code**, managed by the operating system. They are commonly used for:

- Secrets (API keys, passwords)
- Environment-specific settings (dev, test, prod)

They keep configuration separate from code and improve security.

Example

```
import os

db_host = os.getenv("DB_HOST")
```

If the variable is not set, `None` is returned.

Common Trap

? Why should secrets not be hardcoded? Answer: Hardcoding exposes credentials in version control and creates serious security risks.

Command-line arguments vs Environment variables

Command-line Arguments	Environment Variables
Passed at execution time	Set at OS or shell level
Short-lived	Persistent
Control program behavior	Store configuration
Visible in process list	Hidden from command line

Common Trap

? Which one is better for configuration?

Answer: Environment variables — they decouple configuration from code and improve security.

🧩 TOPIC 17 — Logging & Debugging

(logging basics, debugging techniques, REPL usage)

What is logging?

Answer (detailed)

Logging is the process of **recording events that occur during program execution**. It helps developers understand application behavior, diagnose issues, and monitor systems in production. Unlike `print()`, logging is **structured, configurable, and persistent**, making it suitable for real-world systems.

Logs can be:

- Filtered by severity
- Written to files or external systems
- Enabled/disabled per environment

Common Trap

? Why should we not use `print()` in production? Answer: `print()` cannot be filtered, categorized, formatted, or persisted reliably, making monitoring and debugging difficult.

Logging levels in Python

Answer (detailed)

Python's logging framework classifies messages by **severity level**. This allows different verbosity in development vs production environments.

```
Level     Purpose
DEBUG     Detailed diagnostic information
INFO      General application events
WARNING   Unexpected but non-fatal issues
ERROR     Operation failed
CRITICAL  Application/system crash
```

Example

```
import logging

logging.basicConfig(level=logging.INFO)
logging.info("App started")
logging.error("Something failed")
```

Only messages at or above the configured level are shown.

Common Trap

? Why are DEBUG logs sometimes not visible? Answer: Because the configured logging level filters out messages below the threshold.

How is logging better than `print()`?

Answer (detailed)

Logging provides:

- Severity levels
- Timestamps
- File and stream output
- Configurable formatting
- Integration with monitoring tools

It separates **diagnostics from program logic** and scales well in large systems.

Example

```
logging.basicConfig(
    filename="app.log",
    level=logging.INFO,
    format="%(asctime)s - %(levelname)s - %(message)s"
)
```

Logs are now timestamped and stored persistently.

Common Trap

? Does logging slow down applications? Answer: Minimal overhead when used correctly, but excessive debug logging in hot paths can impact performance.

What is debugging?

Answer (detailed)

Debugging is the process of **finding, analyzing, and fixing bugs**. It involves:

- Inspecting variable values
- Tracing execution flow
- Understanding error messages and stack traces

Effective debugging relies on good logs, readable code, and systematic analysis.

What is pdb?

Answer (detailed)

pdb is Python's **built-in interactive debugger**. It allows developers to pause execution, step through code line by line, inspect variables, and control flow.

Example

```
import pdb
pdb.set_trace()
```

Execution stops and opens an interactive debugging prompt.

Common Trap

? Is pdb suitable for production debugging? Answer: No. It is intended for **local development and testing**, not live production systems.

What is a traceback?

Answer (detailed)

A traceback is a report showing the **call stack** when an exception occurs. It lists:

- File names
- Line numbers
- Function calls

Tracebacks help identify **where the error occurred** and **how execution reached that point**.

Common Trap

? Should tracebacks be shown in production? Answer: No. Detailed tracebacks can expose internal logic and sensitive information. They should be logged securely instead.

What is REPL?

Answer (detailed)

REPL stands for **Read–Eval–Print Loop**. It allows developers to run Python code interactively, evaluate expressions, and immediately see results. The Python shell (python) is a REPL environment.

Example

```
python
>>> 2 + 3
```

```
5
```

Common Trap

? Why is REPL useful for debugging?

Answer: It enables quick experimentation and hypothesis testing without running full programs.

🧩 TOPIC 18 — Typing & Code Quality

(type hints, static typing, advanced typing, PEP 8, linters, formatters)

What are type hints in Python?

Answer (detailed)

Type hints allow developers to **annotate expected data types** for variables, function parameters, and return values. They improve readability, documentation, and tooling support, and help catch bugs early through static analysis. Type hints **do not enforce types at runtime** unless additional tools are used.

Example

```
def add(a: int, b: int) -> int:
    return a + b
```

Common Trap

? Does Python raise an error if the wrong type is passed? Answer: No. Type hints are optional and checked only by external tools like `mypy`.

What is static typing?

Answer (detailed)

Static typing means verifying type correctness **before runtime** using analysis tools. Python supports **gradual typing**, allowing developers to add type hints incrementally without breaking existing code. This balances flexibility with safety.

Common Trap

? Is Python statically typed? Answer: No. Python is dynamically typed but supports optional static type checking.

Why use type hints?

Answer (detailed)

Type hints:

- Improve maintainability in large codebases
- Catch bugs earlier via static analysis
- Enhance IDE features (autocomplete, refactoring)
- Improve team collaboration and onboarding

They act as **executable documentation** for developers and tools.

What is `TypedDict`?

Answer (detailed)

`TypedDict` defines the **expected structure of dictionaries** used as structured data. It ensures consistent key usage while keeping dictionaries lightweight and flexible. Commonly used for JSON-like data and API payloads.

Example

```
from typing import TypedDict

class User(TypedDict):
    name: str
    age: int
```

Common Trap

? Why not use a class instead of `TypedDict`? Answer: `TypedDict` avoids object creation overhead and works naturally with dict-based data.

What is `Protocol`?

Answer (detailed)

`Protocol` enables **structural typing**, also known as type-safe duck typing. An object matches a protocol if it has the required methods—inheritance is not required. This improves flexibility while preserving static type safety.

Example

```
from typing import Protocol

class Flyable(Protocol):
    def fly(self) -> None: ...
```

Common Trap

? Difference between inheritance and `Protocol`? Answer: Inheritance enforces a relationship; `Protocol` enforces behavior.

What is `Literal`?

Answer (detailed)

`Literal` restricts a value to **specific allowed constants**. It helps catch invalid values during static type checking, especially for flags or state values.

Example

```
from typing import Literal

def status(s: Literal["open", "closed"]):
    pass
```

Passing any other value will be flagged by type checkers.

What is `Final`?

Answer (detailed)

`Final` indicates that a variable **should not be reassigned**. It is commonly used for constants and configuration values to prevent accidental changes.

Example

```
from typing import Final

PI: Final = 3.14
```

Reassignment will be flagged by static analysis tools.

What is PEP 8?

Answer (detailed)

PEP 8 is Python's **official style guide**. It defines naming conventions, spacing, line length, and code layout rules to improve readability and consistency across projects.

Common Trap

? Is PEP 8 mandatory? Answer: No, but it is strongly recommended and widely enforced in professional codebases.

What are linters and formatters?

Answer (detailed)

- **Linters** analyze code for potential bugs, style violations, and bad practices
- **Formatters** automatically rewrite code to match formatting standards

They improve code quality and enforce consistency across teams.

Examples

- Linters: `flake8`, `pylint`
- Formatters: `black`, `isort`

Common Trap

? Difference between `black` and `flake8`? Answer: `black` reformats code automatically; `flake8` only reports issues.

🧩 TOPIC 19 — Concurrency & Parallelism

(concurrency vs parallelism, threads, processes, async, event loop, race conditions, deadlocks)

What is concurrency?

Answer (detailed)

Concurrency means **making progress on multiple tasks at the same time conceptually**, even if only one task is executing at a given instant. It focuses on **task coordination, responsiveness, and efficient waiting**, not raw speed. Concurrency is especially useful for **I/O-bound workloads** like network calls, disk access, or user input.

Common Trap

? Does concurrency always mean parallel execution? Answer: No. Concurrency can exist on a single CPU via task switching and scheduling.

What is parallelism?

Answer (detailed)

Parallelism means **executing multiple tasks at the same time physically**, usually on multiple CPU cores. It focuses on **performance and speed**, especially for **CPU-bound workloads** like computations or data processing.

Common Trap

? Can Python achieve true parallelism? Answer: Yes — by using **multiprocessing**, not threads.

Difference between concurrency and parallelism

Concurrency	Parallelism
Task switching	Simultaneous execution
Improves responsiveness	Improves performance
Logical structure	Physical execution
Best for I/O-bound work	Best for CPU-bound work

What is multithreading?

Answer (detailed)

Multithreading allows multiple threads to run within the **same process**. Threads **share memory**, which makes communication fast but introduces synchronization challenges. In CPython, threads are limited by the

Global Interpreter Lock (GIL).

Example

```
import threading

def task():
    print("running")

t = threading.Thread(target=task)
t.start()
```

Common Trap

? Why don't threads speed up CPU-heavy tasks in Python? Answer: Because the **GIL allows only one thread to execute Python bytecode at a time**.

What is multiprocessing?

Answer (detailed)

Multiprocessing runs tasks in **separate processes**, each with its own memory and Python interpreter. This bypasses the GIL and enables **true parallelism** on multiple CPU cores. Processes are heavier than threads but ideal for CPU-bound tasks.

Example

```
from multiprocessing import Process

def task():
    print("run")

p = Process(target=task)
p.start()
```

Common Trap

? Why is multiprocessing heavier than threading? Answer: Because processes require separate memory spaces and inter-process communication (IPC).

What is async programming?

Answer (detailed)

Async programming allows handling **many I/O-bound tasks efficiently using a single thread**. It relies on **non-blocking operations** and **cooperative multitasking**, where tasks voluntarily yield control. Async does

not use threads by default and avoids context-switching overhead.

Example

```
import asyncio

async def task():
    await asyncio.sleep(1)
    print("done")

asyncio.run(task())
```

Common Trap

? Is async faster than threading? Answer: Not for CPU-bound tasks. Async excels at **high I/O concurrency**, not computation.

What is the event loop?

Answer (detailed)

The event loop is the **core scheduler** of async programming. It:

- Runs coroutines
- Pauses them at `await`
- Resumes them when I/O is ready

The loop enables thousands of concurrent I/O tasks using a single thread.

Common Trap

? Can there be multiple event loops? Answer: Yes, but **only one event loop per thread**.

What are Futures and Tasks?

Answer (detailed)

- A **Future** represents a result that will be available later
- A **Task** is a Future that wraps a coroutine and schedules it on the event loop

Tasks drive coroutine execution.

Example

```
task = asyncio.create_task(task_func())
```

What is a race condition?

Answer (detailed)

A race condition occurs when **multiple threads or processes access shared data simultaneously** without proper synchronization. The final result depends on execution timing, making bugs **non-deterministic and hard to reproduce**.

Example

```
counter += 1  # unsafe in threads
```

Common Trap

? How do you prevent race conditions? Answer: By using locks, semaphores, atomic operations, or thread-safe data structures.

What is a deadlock?

Answer (detailed)

A deadlock happens when **two or more threads wait indefinitely for each other's resources**. None of the threads can proceed, causing the system to stall.

Common Trap

? How can deadlocks be avoided? Answer:

- Acquire locks in a consistent order
- Minimize lock scope
- Avoid holding multiple locks when possible

What is starvation?

Answer (detailed)

Starvation occurs when a task **never gets CPU time or resources** because other tasks dominate execution. Unlike deadlock, the system continues running—but some tasks never make progress.

🧩 TOPIC 20 — Python Runtime & Memory

(execution model, bytecode, stack vs heap, GC, reference counting, weak references, GIL)

How does Python execute code?

Answer (detailed)

Python source code (`.py`) is first **compiled into bytecode**, which is a platform-independent intermediate form. This bytecode is executed by the **Python Virtual Machine (PVM)**, which interprets each instruction. The CPU does not execute Python code directly; it executes the interpreter that processes bytecode.

Execution flow:

```
.py → bytecode (.pyc) → Python Virtual Machine → Execution
```

Common Trap

? Why do `.pyc` files exist? Answer: They cache compiled bytecode so Python can skip recompilation and **start faster** on subsequent runs.

What is bytecode?

Answer (detailed)

Bytecode is a **low-level instruction set** generated from Python source code. It is not machine code and must be interpreted by the Python VM. Bytecode is **platform independent** but **interpreter dependent** (CPython, PyPy, etc.).

Example

```python
import dis

def add(a, b):
    return a + b

dis.dis(add)
```

This shows the internal bytecode instructions.

Common Trap

? Is bytecode platform dependent? Answer: No. Bytecode is portable across platforms, but specific to the Python interpreter version.

Stack vs Heap

Answer (detailed)

- **Stack**
  - Stores function call frames
  - Holds local variables and execution state
  - Automatically managed (push/pop)
- **Heap**
  - Stores **all Python objects**
  - Managed by Python's memory manager
  - Objects persist until garbage collected

Common Trap

? Where are Python objects stored? Answer: All Python objects live in the **heap**, not on the stack.

What is reference counting?

Answer (detailed)

Reference counting is Python's **primary garbage collection mechanism**. Each object maintains a count of how many references point to it. When the count reaches zero, the object's memory is immediately reclaimed.

Example

```
a = []
b = a
del a
del b   # object is freed
```

Common Trap

? Why is reference counting alone not sufficient? Answer: It cannot detect **circular references**, where objects reference each other.

Garbage Collection (GC)

Answer (detailed)

Python supplements reference counting with a **cyclic garbage collector**. It detects groups of objects that reference each other but are no longer reachable. The GC runs automatically but can be manually controlled.

Common Trap

? Is garbage collection automatic? Answer: Yes. Python runs GC automatically, but it can be inspected or tuned using the `gc` module.

What are weak references?

Answer (detailed)

Weak references allow referencing an object **without increasing its reference count**. They are useful for caches, observers, and memoization systems where objects should not be kept alive artificially.

Example

```
import weakref

class A:
    pass

obj = A()
w = weakref.ref(obj)
```

Common Trap

? When does a weak reference become `None`? Answer: When the original object is garbage collected.

What is the Global Interpreter Lock (GIL)?

Answer (detailed)

The GIL is a mutex that ensures **only one thread executes Python bytecode at a time** in CPython. It simplifies memory management and prevents race conditions inside the interpreter. However, it prevents true parallelism for **CPU-bound multithreaded programs**.

Common Trap

? Why does Python still use the GIL? Answer: To keep memory management thread-safe and the interpreter implementation simpler and faster for single-threaded code.

Is the GIL present in multiprocessing?

Answer (detailed)

No. Each process created via multiprocessing has:

- Its own Python interpreter
- Its own memory space
- Its own GIL

This is why multiprocessing enables **true parallel execution** on multiple CPU cores.

🧩 TOPIC 21 — Metaprogramming & Reflection

(metaclasses, `type()` internals, dynamic class creation, introspection, reflection, `inspect`)

What is metaprogramming?

Answer (detailed)

Metaprogramming is the technique where a program can **inspect, generate, or modify code at runtime**. Python supports metaprogramming because **everything is an object**, including classes, functions, and modules. It is heavily used in frameworks to automate repetitive patterns and enforce conventions dynamically.

Typical use cases:

- ORM models
- Validation frameworks
- API/schema generation

Common Trap

? Where is metaprogramming used in real projects? Answer: Frameworks like Django, SQLAlchemy, Pydantic, FastAPI, and ORMs rely heavily on metaprogramming.

What is `type()` in Python?

Answer (detailed)

`type()` has **two roles** in Python:

1. Returns the type of an object
2. Acts as the **default metaclass** used to create all classes

Internally, **every class is created by calling `type()`**.

Example

```
class A:
    pass

print(type(A))  # <class 'type'>
```

This shows that `A` itself is an object created by `type`.

Common Trap

? Who creates classes in Python? Answer: The metaclass — by default, it is `type`.

Dynamic class creation

Answer (detailed)

Python allows creating classes **dynamically at runtime** using `type()`. This is useful when class definitions depend on configuration, metadata, or external schemas.

Signature of `type()`:

```
type(class_name, base_classes, attributes_dict)
```

Example

```
MyClass = type(
    "MyClass",
    (),
    {"x": 10}
)

obj = MyClass()
print(obj.x)
```

Common Trap

? Why create classes dynamically? Answer: To generate models, APIs, serializers, or schemas dynamically based on runtime data.

What is a metaclass?

Answer (detailed)

A metaclass is a **class that creates other classes**. When Python encounters a class definition, it calls the metaclass to construct the class object. Metaclasses allow control over **class creation**, not object instantiation.

Example

```
class Meta(type):
    def __new__(cls, name, bases, attrs):
        return super().__new__(cls, name, bases, attrs)

class MyClass(metaclass=Meta):
    pass
```

Execution order (important)

```
Class body → Metaclass __new__ → Class object created → Instances created
later
```

Common Trap

? Is a metaclass involved in object creation? Answer: No. Metaclasses run during **class creation**, not when instances are created.

What is introspection?

Answer (detailed)

Introspection is the ability of a program to **examine objects at runtime**. Python allows inspecting:

- Attributes
- Methods
- Types
- Object structure

This enables dynamic behavior and tooling.

Example

```
dir(obj)
hasattr(obj, "name")
getattr(obj, "name", None)
```

Common Trap

? Why is introspection important? Answer: It enables debugging, dynamic frameworks, dependency injection, and runtime validation.

What is reflection?

Answer (detailed)

Reflection goes one step beyond introspection. It allows a program to **dynamically modify or execute code** based on inspected information. Reflection is commonly used to call methods or modify attributes dynamically.

Example

```

```

```
method = getattr(obj, "run")
method()
```

Common Trap

? Difference between introspection and reflection? Answer:

- Introspection → inspect structure
- Reflection → modify or execute dynamically

What is the `inspect` module?

Answer (detailed)

The `inspect` module provides **deep introspection utilities** for live Python objects. It can retrieve:

- Function signatures
- Source code
- Call stacks
- Argument specifications

It is heavily used in tooling and frameworks.

Example

```
import inspect

inspect.getsource(my_function)
```

Common Trap

? Where is `inspect` used in practice?

Answer: Testing frameworks, dependency injection systems, decorators, API validators, and documentation generators.

🧩 TOPIC 22 — Performance & Optimization

(time complexity, space complexity, bottlenecks, profiling concepts)

What is time complexity?

Answer (detailed)

Time complexity describes how an algorithm's **execution time grows as input size increases**. It allows comparing algorithms **independently of hardware, language, or implementation details**. Time complexity is usually expressed using **Big-O notation**, which focuses on worst-case growth.

Common Trap

? Does Big-O represent actual execution time? Answer: No. Big-O describes **growth rate**, not real runtime in seconds.

What is space complexity?

Answer (detailed)

Space complexity measures how much **extra memory (auxiliary space)** an algorithm requires relative to input size. It includes memory used by variables, data structures, recursion stack, and temporary objects. The input itself is usually **not counted**.

Common Trap

? Is recursion worse for space complexity? Answer: Yes. Each recursive call consumes stack space, increasing memory usage.

What is Big-O notation?

Answer (detailed)

Big-O notation expresses the **upper bound** of an algorithm's time or space usage. It ignores constants and lower-order terms to focus on scalability as input grows large. Big-O helps reason about algorithm efficiency at scale.

Common examples

Complexity	Example
O(1)	Dictionary lookup
O(n)	Single loop
O(log n)	Binary search

Complexity	Example
$O(n^2)$	Nested loops

Common Trap

? Why are constants ignored in Big-O? Answer: Because constants do not affect long-term scalability as input size increases.

What are performance bottlenecks?

Answer (detailed)

A performance bottleneck is the **slowest or most resource-heavy part of a system** that limits overall performance. Optimizing code outside the bottleneck usually has **little or no impact**. Effective optimization focuses on **hot paths**, not random parts of code.

Common Trap

? What is the first rule of optimization? Answer: Do not optimize without **measurement**.

How do you identify performance issues?

Answer (detailed)

Performance issues are identified using **profiling**, not guesswork. Profilers measure where time and memory are actually spent during execution. They help pinpoint slow functions and inefficient patterns.

Common tools

```
cProfile        # function-level profiling
timeit          # micro-benchmarks
line_profiler   # line-by-line timing
```

Common Trap

? Why not rely on intuition for optimization? Answer: Because intuition is often wrong; profiling provides **objective data**.

Python performance optimization techniques

Answer (detailed)

Python optimization focuses on **reducing interpreter overhead**, not micro-tuning. Common techniques include:

- Using built-in functions and libraries
- Choosing the right data structures
- Avoiding unnecessary loops and object creation
- Using generators for large data
- Reducing repeated computations

Common Trap

? Why are built-in functions faster?

Answer: They are implemented in **C**, bypassing Python interpreter overhead.

🧩 TOPIC 23 — Security (Conceptual)

(hashing, encryption, salting, secure coding practices, Python security pitfalls)

What is hashing?

Answer (detailed)

Hashing is a **one-way cryptographic process** that converts input data into a fixed-length string (hash). It is designed so that the original input **cannot be reconstructed** from the hash. Hashing is mainly used for **password storage**, integrity checks, and data fingerprinting.

Example

```
import hashlib

hashlib.sha256(b"password").hexdigest()
```

Common Trap

? Can hashed data be decrypted? Answer: No. Hashing is irreversible by design. You can only **compare hashes**, not recover the original data.

What is encryption?

Answer (detailed)

Encryption is a **two-way process** used to protect sensitive data. Encrypted data can be converted back to its original form using a **secret key**. Encryption is used when data must be **read later**, such as files, tokens, or messages.

Common Trap

? When should encryption be used instead of hashing? Answer: When the original data must be retrieved later (e.g., API keys, confidential files).

Difference between hashing and encryption

Hashing	Encryption
One-way	Two-way
Irreversible	Reversible
Used for passwords	Used for secrets

Hashing	Encryption
No key required	Requires key

Common Trap

? Why should passwords never be encrypted? Answer: If encryption keys are compromised, passwords can be decrypted. Hashing ensures passwords are **never stored in retrievable form**.

What is salting?

Answer (detailed)

Salting means adding a **random value** to a password before hashing it. This ensures that identical passwords produce **different hashes**. Salting protects against **rainbow table attacks** and mass credential cracking.

Common Trap

? Why is salting important even with strong hashing algorithms? Answer: Without salt, identical passwords generate identical hashes, making large-scale attacks much easier.

What are common Python security pitfalls?

Answer (detailed)

Common Python security issues often come from unsafe defaults or misuse of features:

- Hardcoding secrets (API keys, passwords)
- Unsafe `pickle` deserialization
- SQL injection due to string-based queries
- Command injection via `os.system` or `subprocess`
- Insecure dependency usage

These issues typically arise from **lack of validation and trust boundaries**.

Common Trap

? Why is `pickle` considered unsafe? Answer: `pickle` can execute **arbitrary code** during deserialization. Loading untrusted pickle data can lead to **remote code execution**.

What are secure coding practices in Python?

Answer (detailed)

Secure coding means **designing software with security in mind from the start**. Key practices include:

- Validating all user input
- Never trusting external data

- Using parameterized queries
- Avoiding unsafe deserialization
- Managing secrets securely
- Keeping dependencies updated

Security is not a feature—it's a **process**.

Common Trap

? Is security only the responsibility of security teams? Answer: No. Most vulnerabilities originate in application code, so developers play a critical role in security.

🧩 TOPIC 24 — System Design & Distributed Systems (Conceptual)

(CAP, ACID, BASE, consistency, idempotency, scalability, availability, fault tolerance, locking)

What is a distributed system?

Answer (detailed)

A distributed system consists of **multiple independent machines** that coordinate over a network to appear as a single system. Each node runs independently, communicates via messages, and may fail independently. Failures, latency, and partial outages are **expected**, not exceptional.

Designing distributed systems requires assuming:

- Networks are unreliable
- Nodes can crash
- Messages can be delayed or duplicated

Common Trap

? Why are distributed systems difficult to build? Answer: Because failures are **partial and unpredictable**. You must manage consistency, latency, replication, and recovery at the same time.

What is the CAP Theorem?

Answer (detailed)

The CAP theorem states that a distributed system can guarantee **only two out of three** properties at the same time:

Property	Meaning
Consistency	All nodes see the same data
Availability	Every request receives a response
Partition Tolerance	System works despite network splits

When a network partition occurs, the system must choose between **consistency** and **availability**.

Common Trap

? Which CAP property is mandatory? Answer: Partition tolerance — because network failures are unavoidable in distributed systems.

CP vs AP systems

CP (Consistency + Partition Tolerance)

- May reject requests during partitions
- Guarantees strong consistency
- Prioritizes correctness over availability

Examples: traditional relational DB clusters, Zookeeper

AP (Availability + Partition Tolerance)

- Always responds, even during partitions
- Allows temporary inconsistency
- Uses eventual consistency

Examples: Cassandra, DynamoDB

Common Trap

? Is CA possible? Answer: No in distributed systems. CA is only possible in **single-node systems** where partitions do not exist.

What is ACID?

Answer (detailed)

ACID defines guarantees for **transactional databases** to ensure correctness:

Property	Meaning
Atomicity	All or nothing
Consistency	Valid state transitions
Isolation	Concurrent safety
Durability	Data persists after commit

ACID is common in **relational databases**.

Common Trap

? Does ACID imply strong consistency? Answer: Yes. ACID systems typically provide strong consistency guarantees.

What is BASE?

Answer (detailed)

BASE is a model used by **distributed NoSQL systems** that favors scalability and availability:

BASE	Meaning

BASE	Meaning
Basically Available	System responds
Soft State	State may change
Eventual Consistency	Data syncs over time

BASE trades strict correctness for **scalability and fault tolerance**.

Common Trap

? Why does BASE exist if ACID already exists? Answer: Because ACID does not scale well in highly distributed, globally replicated systems.

What is a consistency model?

Answer (detailed)

A consistency model defines **when and how data updates become visible** to users. Different systems choose different models depending on correctness and performance needs.

Common models

- Strong consistency
- Eventual consistency
- Read-your-writes
- Causal consistency

Common Trap

? Is eventual consistency wrong? Answer: No. It's acceptable when temporary inconsistency is tolerable (e.g., likes, feeds, counters).

What is idempotency?

Answer (detailed)

An operation is **idempotent** if repeating it produces the same result. This is critical in distributed systems because retries are common due to timeouts and failures.

Example

```
DELETE /user/10
```

Calling it multiple times has the same effect.

Common Trap

? Why is idempotency critical in APIs? Answer: Because retries can cause duplicate requests. Idempotency prevents duplicate side effects.

Stateless vs Stateful systems

Stateless systems

- Do not store client session data
- Easy to scale horizontally
- Common in microservices

Stateful systems

- Store session or client data
- Harder to scale
- Require replication or sticky sessions

Common Trap

? Why do stateless systems scale better? Answer: Because any request can be handled by any server without dependency on prior state.

Scalability: Vertical vs Horizontal

Vertical scaling (scale up)

- Add CPU/RAM to one machine
- Limited growth
- Single point of failure

Horizontal scaling (scale out)

- Add more machines
- Highly scalable
- Fault tolerant
- Cloud-friendly

Common Trap

? Why is horizontal scaling preferred in cloud systems? Answer: It enables elasticity, redundancy, and cost-effective scaling.

High Availability (HA)

Answer (detailed)

High availability ensures the system remains **operational during failures**. It is achieved through:

- Redundancy
- Replication
- Load balancing

HA focuses on **minimizing downtime**, not eliminating failures.

Common Trap

? Is high availability the same as zero downtime? Answer: No. HA minimizes downtime but cannot guarantee zero failure.

Fault Tolerance

Answer (detailed)

Fault tolerance means the system continues operating **correctly even when components fail**. It requires:

- Failure detection
- Automatic recovery
- Data replication

Common Trap

? Difference between HA and fault tolerance? Answer: HA minimizes downtime; fault tolerance ensures correctness despite failures.

Distributed locking

Answer (detailed)

Distributed locks ensure **only one node performs a critical operation** at a time. They are needed when multiple services share a resource.

Common implementations

- Redis Redlock
- Zookeeper locks

Common Trap

? Why are distributed locks hard? Answer: Because clock drift, network delays, crashes, and partial failures can leave locks in inconsistent states.

🧩 TOPIC 25 — Design Principles & Design Patterns

(SOLID, patterns, dependency injection, coupling, clean code)

What are design principles?

Answer (detailed)

Design principles are **guidelines for structuring software** so it remains maintainable, scalable, and adaptable over time. They are not strict rules, but lessons learned from real-world system failures. Applying them helps reduce tight coupling, improve readability, and prevent frequent rewrites as systems grow. They become critical in large codebases and multi-team environments.

Common Trap

? Are design principles mandatory to follow? Answer: No. They are guiding concepts, not laws. Blindly applying them can overcomplicate simple systems. Good engineers apply principles **when complexity justifies structure**.

What is SOLID?

Answer (detailed)

SOLID is a set of **five object-oriented design principles** that improve flexibility and maintainability. They help systems evolve without breaking existing behavior. SOLID promotes separation of concerns, loose coupling, and extensibility. It is most valuable in **large, long-lived systems**, not small scripts.

Single Responsibility Principle (SRP)

Answer (detailed)

SRP states that a class should have **only one reason to change**. A responsibility refers to a **single axis of change**, not size or method count. Violating SRP leads to fragile, tightly coupled code. Following SRP improves testability and clarity.

Common Trap

? Is one responsibility equal to one method? Answer: No. A class may have many methods as long as they serve the same responsibility. The focus is **cohesion**, not class size.

Open–Closed Principle (OCP)

Answer (detailed)

OCP states that software should be **open for extension but closed for modification**. New behavior should be added without changing existing, tested code. This minimizes regression bugs and improves system

stability. It is commonly achieved using abstraction, inheritance, or composition.

Common Trap

? Why is modifying existing code risky? Answer: Because existing code may already be in production. Changes can introduce hidden side effects. Extending behavior avoids breaking proven logic.

Liskov Substitution Principle (LSP)

Answer (detailed)

LSP states that **objects of a subclass should be usable wherever the base class is expected**. Derived classes must preserve the behavior and expectations of base classes. Violating LSP breaks polymorphism and causes runtime surprises. It ensures inheritance represents a true “is-a” relationship.

Common Trap

? What happens if LSP is violated? Answer: Code relying on base-class behavior starts failing unexpectedly. Inheritance hierarchies become fragile and unpredictable.

Interface Segregation Principle (ISP)

Answer (detailed)

ISP states that clients should not depend on **interfaces they do not use**. Large interfaces should be split into smaller, focused ones. This reduces unnecessary dependencies and improves flexibility. It is especially important in large or evolving systems.

Common Trap

? Why are large interfaces dangerous? Answer: They force implementations to provide irrelevant methods, leading to dummy code and poor design. Smaller interfaces are easier to maintain and evolve.

Dependency Inversion Principle (DIP)

Answer (detailed)

DIP states that **high-level modules should not depend on low-level modules**. Both should depend on abstractions. This decouples business logic from implementation details. It enables easier testing, replacement, and evolution of components.

Common Trap

? How is DIP implemented in real systems? Answer: Through interfaces, dependency injection, and inversion of control (often managed by frameworks).

What is Dependency Injection (DI)?

Answer (detailed)

Dependency Injection is a technique where dependencies are **provided from outside**, rather than created internally. It reduces coupling and improves testability. Objects become easier to mock, replace, and extend. DI is foundational in scalable architectures and frameworks.

Common Trap

? Why not create dependencies inside the class? Answer: It tightly couples code to concrete implementations, making testing and future changes difficult.

Tight coupling vs Loose coupling

Tight coupling

- Hard to change
- Hard to test
- Rigid and fragile system

Loose coupling

- Flexible
- Testable
- Easier to evolve and scale

Common Trap

? Why does loose coupling matter in microservices? Answer: Services evolve independently. Tight coupling causes cascading failures. Loose coupling improves resilience and scalability.

Common Design Patterns (High-Level)

Singleton

Ensures only one instance exists (use carefully).

Factory

Encapsulates object creation logic.

Strategy

Allows changing behavior at runtime via composition.

Observer

Notifies multiple subscribers when state changes.

Common Trap

? Are design patterns still relevant today?

Answer: Yes—but many are built into frameworks. Patterns teach **problem-solving approaches**, not copy-paste code.

Clean Code Principles

Answer (detailed)

Clean code emphasizes **readability, simplicity, and clarity**. It favors explicitness over cleverness and long-term maintainability over short-term speed. Clean code is easier to debug, review, and extend. It significantly reduces technical debt over time.

Common Trap

? Is clean code subjective?

Answer: Partially—but many principles are universal. Consistency and clarity matter more than personal style.

🧩 TOPIC 26 — Common Python Interview Questions & Pitfalls

(Very frequently asked, especially in senior interviews)

List vs Tuple

Answer (refined)

Lists are **mutable**, meaning their contents can be changed after creation. Tuples are **immutable**, so their structure cannot be modified once created. Tuples are slightly faster, use less memory, and can be dictionary keys if they contain only immutable values. Lists are preferred when data needs to change.

Common Trap

? If tuples are immutable, how can they contain mutable objects?

Answer: Tuple immutability applies to the **references**, not the objects themselves. You cannot reassign elements of a tuple, but if a tuple contains a mutable object (like a list), that object can still be modified. This is a very common interview pitfall.

`==` vs `is`

Answer (refined)

`==` checks **value equality** (do the contents match). `is` checks **object identity** (do both variables point to the same object in memory). Using `is` for value comparison is a common mistake.

Common Trap

? Why does `a is b` sometimes return True for integers?

Answer: Python caches small integers (typically `-5` to `256`) for performance. Variables pointing to these values reference the same object. This is an implementation detail and should not be relied upon.

Mutable default arguments

Answer (refined)

Default arguments are evaluated **once at function definition time**, not at call time. If a mutable object is used as a default, it is **shared across calls**, leading to unexpected behavior. This is one of the most common Python bugs.

Example

```
def add_item(item, items=[]):
    items.append(item)
    return items
```

Common Trap

? Why does the list keep growing across calls?

Answer: Because the same list object is reused every time. The default value is created once, so state persists across function calls.

✅ **Correct pattern**

```
def add_item(item, items=None):
    if items is None:
        items = []
    items.append(item)
    return items
```

When should you use generators?

Answer (refined)

Generators should be used for **large datasets, streams, or pipelines**. They generate values lazily and consume minimal memory. They are ideal for file processing, data streaming, and infinite sequences.

Common Trap

? When should generators NOT be used?

Answer: When data must be reused multiple times or accessed randomly. Generators are exhausted after one iteration.

When should you use decorators?

Answer (refined)

Decorators are useful for adding **cross-cutting behavior** without changing core logic. Common use cases include logging, authentication, caching, retries, and access control. They promote DRY, but overuse can reduce readability.

Common Trap

? Can decorators make debugging harder?

Answer: Yes. They add abstraction layers. Without `functools.wraps`, stack traces and function metadata become misleading.

Why is Python considered slow?

Answer (refined)

Python is slower due to:

- Dynamic typing
- Interpreter overhead
- Runtime type checks
- GIL limiting CPU-bound threading

However, performance is often dominated by I/O, not CPU.

Common Trap

? Then why is Python used in large systems?

Answer: Because most real systems are I/O-bound. Python integrates with optimized C/C++ libraries and offers high developer productivity.

Why is Python still “fast enough”?

Answer (refined)

Python offloads heavy work to optimized native libraries (NumPy, databases, networking stacks). Async programming and multiprocessing improve throughput. Python excels as a **business logic and orchestration layer**, not raw computation.

Common Trap

? Is Python suitable for everything?

Answer: No. Real-time systems and heavy CPU workloads often require lower-level languages. Python shines in backend systems, automation, data pipelines, and APIs.

🧩 TOPIC 27 — Python Behavior & Internals (Frequently Asked but Tricky)

*(Questions that test **real understanding**, not memorization)*

1️⃣ Why are Python strings immutable?

Explained Answer

Strings are immutable to ensure **memory efficiency, safety, and performance**. Immutable strings can be safely shared across the program without fear of accidental modification. This enables optimizations like **string interning** and makes strings hashable (usable as dictionary keys).

Changing a string actually creates a **new object**, not a modification.

Interview Follow-up Trap

? *Then how does `s += "x"` work?*

Answer: It creates a **new string object** and rebinds the variable. The original string remains unchanged.

2️⃣ Why are dictionary lookups so fast?

Explained Answer

Dictionaries are implemented using **hash tables**. Keys are converted into hash values, which map directly to memory locations. This allows **average O(1)** lookup, insert, and delete operations.

Performance only degrades in rare cases of heavy hash collisions.

Interview Follow-up Trap

? *Are dictionary lookups always O(1)?*

Answer: No. Worst case is O(n), but Python's hashing strategy makes this extremely rare.

3️⃣ What happens when you pass a list to a function?

Explained Answer

Python passes **object references**, not copies. When you pass a list, both the caller and function reference the **same object**. Mutating the list inside the function affects the original list.

Example

```
def modify(lst):
    lst.append(10)
```

```
a = [1, 2]
modify(a)
print(a)  # [1, 2, 10]
```

Interview Follow-up Trap

? *How do you avoid modifying the original list?*

Answer: Pass a copy using `lst[:]` or `list(lst)`.

4️⃣ What is Python's object model?

Explained Answer

In Python, **everything is an object**:

- Integers
- Functions
- Classes
- Modules

Each object has:

- Identity (`id`)
- Type (`type`)
- Value

This uniform object model enables dynamic behavior and metaprogramming.

Interview Follow-up Trap

? *Is a class an object?*

Answer: Yes. Classes are objects created by the metaclass `type`.

5️⃣ Why does Python allow dynamic typing?

Explained Answer

Dynamic typing improves **developer productivity and flexibility**. Types are bound to objects, not variable names. This allows rapid development and easier refactoring.

Trade-off: errors appear at runtime instead of compile time.

Interview Follow-up Trap

? *How do large Python codebases handle this risk?*

Answer: Using type hints, static analysis tools, tests, and disciplined design.

6️⃣ What is name mangling in Python?

Explained Answer

Name mangling occurs when attributes start with double underscores (`__var`). Python rewrites them internally as `_ClassName__var` to avoid accidental overrides in subclasses.

It is **not true privacy**, just namespace protection.

Example

```
class A:
    def __init__(self):
        self.__x = 10
```

Internally becomes `_A__x`.

Interview Follow-up Trap

? *Can you still access mangled variables?*

Answer: Yes, but you shouldn't. It breaks encapsulation.

7️⃣ Why does `list.append()` return None?

Explained Answer

Methods that **modify objects in-place** return `None` by design. This prevents chaining and avoids confusion between mutation and value creation.

It enforces a clear distinction:

- Mutation → no return
- Computation → returns value

Interview Follow-up Trap

? *Is this a Python limitation?*

Answer: No. It's a **design choice** for clarity and safety.

8️⃣ What is truthy and falsy in Python?

Explained Answer

Python evaluates objects in boolean context using truthiness rules.

Falsy values include:

- `False`
- `None`
- `0`
- `""`
- `[]`, `{}`, `()`

Everything else is truthy unless `__bool__()` or `__len__()` says otherwise.

Interview Follow-up Trap

? *Can custom objects be truthy or falsy?*

Answer: Yes. By defining `__bool__()` or `__len__()`.

9️⃣ Why does Python allow multiple return values?

Explained Answer

Python returns **a single tuple**, not multiple values. Tuple unpacking makes it appear like multiple returns.

This improves readability and avoids wrapper objects.

Example

```
def f():
    return 1, 2

a, b = f()
```

Interview Follow-up Trap

? *Is this efficient?*

Answer: Yes. Tuple packing/unpacking is optimized in CPython.

🔟 Why are Python exceptions considered expensive?

Explained Answer

Exceptions interrupt normal execution flow. They involve stack unwinding and traceback creation, which is costly. Exceptions should handle **exceptional cases**, not regular control flow.

Interview Follow-up Trap

? *Is `try/except` slower than `if`?*

Answer: Yes—if exceptions are raised frequently. But `try` blocks themselves are cheap when no exception occurs.

1️⃣ Why is `None` a singleton in Python?

Explained Answer

`None` represents the **absence of a value**, so having multiple `None` objects would be meaningless. Using a single instance:

- Saves memory
- Allows fast identity checks (`x is None`)
- Ensures semantic consistency

That's why you should always check `None` using `is`, not `==`.

2️⃣ Why is `__init__` not a constructor?

Explained Answer

The actual object is created by `__new__`, not `__init__`. `__init__` only **initializes an already created object**. This separation allows:

- Immutable object creation
- Custom memory allocation
- Advanced class behavior

Most developers never override `__new__`, but it's essential internally.

3️⃣ Why does Python prefer composition over inheritance?

Explained Answer

Inheritance creates **tight coupling** and fragile hierarchies. Composition:

- Is more flexible
- Avoids deep inheritance trees
- Allows runtime behavior changes

Python's dynamic nature and duck typing make composition a safer default.

4️⃣ Why does `sorted()` return a new list but `list.sort()` doesn't?

Explained Answer

`sorted()` is a **functional-style operation** — it does not modify inputs. `list.sort()` is an **in-place mutation** for performance reasons.

This design:

- Avoids unnecessary copying

- Makes mutation explicit
- Prevents confusion in chained calls

5️⃣ Why is recursion discouraged in Python?

Explained Answer

Python does not optimize tail recursion and has a **limited recursion depth**. Recursive calls:

- Consume stack memory
- Are slower than loops
- Risk stack overflow

Iteration is usually safer and faster in Python.

6️⃣ Why are global variables dangerous?

Explained Answer

Global variables:

- Break encapsulation
- Make debugging difficult
- Introduce hidden dependencies
- Cause concurrency issues

They make programs harder to reason about and scale.

7️⃣ Why does Python not support method overloading?

Explained Answer

Python uses **dynamic typing**, so method signatures are not enforced at compile time. Method overloading is replaced by:

- Default arguments
- `*args` / `**kwargs`
- Dynamic behavior

This keeps the language simpler and more flexible.

8️⃣ Why does `list.append()` return `None`?

Explained Answer

Methods that **mutate objects in place** return `None` intentionally. This prevents confusing code like:

```
a = a.append(10)  # ×
```

It enforces clarity between **mutation** and **value creation**.

9️⃣ Why is `bool` a subclass of `int`?

Explained Answer

Booleans were added later to Python. Making `bool` a subclass of `int` preserved backward compatibility.

This is why:

```
True == 1
False == 0
```

🔟 Why does Python allow monkey patching?

Explained Answer

Python favors **flexibility and dynamism** over strict control. Monkey patching enables:

- Testing
- Runtime fixes
- Framework extensibility

However, it should be used sparingly because it can make systems unpredictable.

1️⃣1️⃣ Why does Python raise errors at runtime, not compile time?

Explained Answer

Python is **interpreted and dynamically typed**. Types are resolved during execution, not beforehand. This allows flexibility but shifts error detection to runtime.

Large systems compensate with:

- Type hints
- Static analyzers
- Tests

1️⃣2️⃣ Why does Python not have private variables?

Explained Answer

Python follows the philosophy:

> *"We are all consenting adults here."*

Privacy is enforced by **convention**, not restriction. This keeps the language simple and flexible.

1️⃣3️⃣ Why is `id()` not a memory address guarantee?

Explained Answer

`id()` is guaranteed to be **unique during an object's lifetime**, not permanent. CPython often uses memory addresses, but this is an implementation detail.

Never rely on `id()` for persistence or identity beyond runtime.

1️⃣4️⃣ Why does Python prefer readability over performance?

Explained Answer

Python's design philosophy prioritizes:

- Readability
- Maintainability
- Developer productivity

Performance-critical code is expected to:

- Use optimized libraries
- Be written in C/C++ extensions

1️⃣5️⃣ Why does Python have a GIL?

Explained Answer

The GIL simplifies:

- Memory management
- Garbage collection
- Thread safety

It improves single-threaded performance and interpreter simplicity, even though it limits CPU-bound threading.

🧠 Python Interview Questions (4–5 Years Experience)

(Conceptual + Practical + "Why" oriented)

1️⃣ What is the difference between shallow copy and deep copy?

Answer

A **shallow copy** creates a new container but keeps references to the same nested objects. A **deep copy** recursively copies all nested objects, creating a fully independent structure.

Example

```
import copy
a = [[1, 2], [3, 4]]
b = copy.copy(a)
c = copy.deepcopy(a)
```

- Modifying `b[0][0]` affects `a`
- Modifying `c[0][0]` does not

2️⃣ How does Python manage memory?

Answer

Python uses:

- **Reference counting** as the primary mechanism
- **Cyclic garbage collector** to handle circular references

Objects are allocated on the heap, while stack frames manage function execution.

3️⃣ What is the difference between `__new__` and `__init__`?

Answer

- `__new__` creates the object
- `__init__` initializes the object

`__new__` is used mainly when dealing with immutable types or custom object creation logic.

4️⃣ What is GIL and why does it exist?

Answer

The **Global Interpreter Lock (GIL)** ensures only one thread executes Python bytecode at a time. It simplifies memory management and makes single-threaded programs faster. However, it limits CPU-bound multithreading.

5️⃣ Difference between multithreading, multiprocessing, and async?

Answer

Model	Best For
Multithreading	I/O-bound tasks
Multiprocessing	CPU-bound tasks
Async	High I/O concurrency

Each model solves a different performance problem.

6️⃣ What happens internally when you import a module?

Answer

Python:

1. Checks `sys.modules` cache
2. Searches paths in `sys.path`
3. Loads and executes module once
4. Caches it for future imports

This prevents duplicate execution.

7️⃣ Why are Python dictionaries unordered (historically)?

Answer

Dictionaries were hash-table based, optimized for fast lookup. Ordering was not guaranteed until **Python 3.7**, where insertion order became language-guaranteed.

8️⃣ What is monkey patching and when is it dangerous?

Answer

Monkey patching modifies objects at runtime. It's dangerous because it:

- Breaks encapsulation
- Causes unexpected behavior
- Makes debugging harder

Acceptable only in controlled environments like testing.

9️⃣ Explain Python's iterator protocol

Answer

An object is iterable if it implements `__iter__()` and `__next__()`. Iteration continues until `StopIteration` is raised.

This protocol powers `for` loops internally.

🔟 Why is `None` preferred over empty values?

Answer

`None` explicitly represents **absence of value**. Empty values like `[]` or `""` can be valid data. Using `None` avoids ambiguity.

1️⃣1️⃣ How does exception handling impact performance?

Answer

Raising exceptions is expensive due to stack unwinding. Use exceptions for **exceptional cases**, not normal flow. `try` blocks themselves are cheap if no exception occurs.

1️⃣2️⃣ What is a closure? Why is it useful?

Answer

A closure is a function that **remembers variables from its enclosing scope**. Closures are useful for:

- Data hiding
- Function factories
- Callback-based designs

1️⃣3️⃣ What is the difference between `@staticmethod` and `@classmethod`?

Answer

Static Method	Class Method
No `self` or `cls`	Receives `cls`
Utility logic	Works with class state

Use `classmethod` when class-level behavior is needed.

1️⃣4️⃣ How do you write thread-safe code in Python?

Answer

Use:

- Locks (`threading.Lock`)
- Thread-safe data structures
- Avoid shared mutable state

For CPU-bound work, prefer multiprocessing.

1️⃣5️⃣ What is idempotency and why is it important?

Answer

An operation is idempotent if repeating it produces the same result. It's critical in APIs because retries are common in distributed systems.

1️⃣6️⃣ How do you profile Python code?

Answer

Common tools:

- `cProfile` – function-level profiling
- `timeit` – micro benchmarking
- `line_profiler` – line-level analysis

Always measure before optimizing.

1️⃣7️⃣ What are weak references used for?

Answer

Weak references allow objects to be referenced without preventing garbage collection. They are commonly used in:

- Caching
- Observers
- Memory-sensitive systems

1️⃣8️⃣ Why should passwords be hashed, not encrypted?

Answer

Encryption is reversible. Hashing is one-way. Passwords should never be recoverable, only verifiable.

1️⃣9️⃣ How does Python support polymorphism without interfaces?

Answer

Python uses **duck typing**. Objects are considered compatible based on behavior, not inheritance.

This makes code flexible and expressive.

2️⃣0️⃣ What are common causes of memory leaks in Python?

Answer

- Circular references with `__del__`
- Global references
- Caches without eviction
- Unclosed resources

Monitoring tools help detect leaks early.

🧠 Python Interview Questions (4–5 Years) — Part 2

(Depth, Internals, Real-World Scenarios)

2️⃣1️⃣ What is the difference between `__repr__` and `__str__`?

Answer

- `__str__` is meant for **end users** (readable output).
- `__repr__` is meant for **developers** (unambiguous, debugging-friendly).

If `__str__` is not defined, Python falls back to `__repr__`.

2️⃣2️⃣ Why is `with` preferred over try/finally?

Answer

The `with` statement:

- Automatically handles setup and cleanup
- Guarantees resource release even on exceptions
- Produces cleaner, safer code

It reduces human error and improves readability.

2️⃣3️⃣ How does Python handle function overloading?

Answer

Python does **not support traditional method overloading**. Instead, it uses:

- Default arguments
- `*args` and `**kwargs`
- Dynamic type checking

This keeps the language flexible and simple.

2️⃣4️⃣ What happens if you modify a list while iterating over it?

Answer

Modifying a list during iteration can:

- Skip elements
- Cause unexpected behavior
- Lead to logical bugs

Correct approaches:

- Iterate over a copy
- Use list comprehensions
- Build a new list

2️⃣5️⃣ What is the difference between `deepcopy()` and slicing?

Answer

- Slicing (`lst[:]`) creates a **shallow copy**
- `deepcopy()` recursively copies all nested objects

Use slicing for flat lists, deepcopy for nested structures.

2️⃣6️⃣ Why is `finally` always executed?

Answer

The `finally` block guarantees **cleanup code execution**, even if:

- An exception is raised
- A return statement is executed

It ensures resource safety and predictable behavior.

2️⃣7️⃣ What is the difference between `map()` and list comprehension?

Answer

List comprehensions are:

- More readable
- More Pythonic
- Easier to debug

`map()` may be slightly faster for simple functions but reduces readability.

2️⃣8️⃣ Why is Python considered strongly typed but dynamically typed?

Answer

Python is:

- **Strongly typed** → no implicit type coercion (`"1" + 1` fails)
- **Dynamically typed** → type checking occurs at runtime

These concepts are orthogonal.

2️⃣9️⃣ How does Python resolve variable names?

Answer

Python uses the **LEGB rule**:

1. Local
2. Enclosing
3. Global
4. Built-in

This determines where a variable is searched.

3️⃣0️⃣ What is the difference between `set` and `frozenset`?

Answer

set	frozenset
Mutable	Immutable
Not hashable	Hashable
Cannot be dict key	Can be dict key

Use `frozenset` when immutability is required.

3️⃣1️⃣ What are Python context managers used for besides files?

Answer

Context managers are used for:

- Thread locks
- Database connections
- Network sockets
- Timing code execution

They guarantee safe setup and teardown.

3️⃣2️⃣ Why should `eval()` be avoided?

Answer

`eval()` executes arbitrary code. It introduces:

- Severe security risks
- Debugging difficulties

Safer alternatives include:

- `ast.literal_eval()`
- Explicit parsing logic

3️⃣3️⃣ What is the difference between `@property` and normal methods?

Answer

`@property` allows:

- Method access like an attribute
- Encapsulation with controlled access
- Future logic addition without API changes

It improves maintainability.

3️⃣4️⃣ What happens when an exception is raised?

Answer

Python:

1. Stops normal execution
2. Searches for an exception handler
3. Unwinds the call stack
4. Executes `finally` blocks
5. Terminates if unhandled

This process is called **stack unwinding**.

3️⃣5️⃣ What is Python's import shadowing problem?

Answer

Import shadowing occurs when:

- A file name conflicts with standard modules (`json.py`, `sys.py`)

Python imports the local file instead of the standard library.

3️⃣6️⃣ Why are generators preferred over lists in pipelines?

Answer

Generators:

- Use less memory
- Support streaming
- Enable lazy evaluation

They are ideal for large or infinite data flows.

3️⃣7️⃣ What is the difference between `break`, `continue`, and `pass`?

Answer

Keyword	Behavior
break	Exits loop
continue	Skips iteration
pass	Does nothing

3️⃣8️⃣ Why is `__slots__` used?

Answer

`__slots__`:

- Reduces memory usage
- Prevents dynamic attribute creation
- Improves attribute access speed

Used in memory-sensitive applications.

3️⃣9️⃣ What is the difference between `os.system` and `subprocess`?

Answer

- `os.system` is simple but unsafe
- `subprocess` is secure, flexible, and recommended

Always prefer `subprocess`.

4️⃣0️⃣ What is the difference between logging and exceptions?

Answer

- **Exceptions** handle errors
- **Logging** records events

Exceptions should not replace logging, and vice versa.

🧠 Python Interview Questions (4–5 Years) — Part 3

(Practical, Internals, Production-oriented)

4️⃣1️⃣ What is the difference between `copy.copy()` and assignment (`=`)?

Answer

Assignment (`=`) does **not create a new object**. It only creates a new reference to the same object.
`copy.copy()` creates a **new outer object**, but inner objects are still shared (shallow copy).

```
a = [1, [2, 3]]
b = a          # same object
c = copy.copy(a)  # new list, shared inner list
```

4️⃣2️⃣ Why does Python have both `list()` and `[]`?

Answer

Both create lists, but:

- `[]` is **faster and preferred** (literal syntax)
- `list()` is useful when converting iterables

```
list("abc")  # ['a', 'b', 'c']
```

Literal syntax is optimized at bytecode level.

4️⃣3️⃣ What is the difference between `range` and `xrange`?

Answer

- In Python 3, **only `range` exists**
- It behaves like `xrange` from Python 2 (lazy, memory efficient)

`range` does **not store all values**, it generates them on demand.

4️⃣4️⃣ Why does Python raise `UnboundLocalError`?

Answer

Python treats a variable as **local** if it is assigned anywhere in the function. If you try to read it before assignment, Python raises `UnboundLocalError`.

```
x = 10
def f():
    print(x)
    x = 5
```

Fix using `global` or `nonlocal`.

4️⃣5️⃣ What is the difference between `global` and `nonlocal`?

Answer

- `global` → refers to module-level variable
- `nonlocal` → refers to enclosing function variable

`nonlocal` is used in closures.

4️⃣6️⃣ Why are Python lambdas limited to one expression?

Answer

Lambdas are designed for **simple, inline expressions**, not complex logic. They improve readability for short operations but discourage hidden complexity.

Use `def` for anything non-trivial.

4️⃣7️⃣ What is method resolution order (MRO)?

Answer

MRO defines the order Python follows to search for methods in inheritance hierarchies. Python uses **C3 linearization** to resolve ambiguity in multiple inheritance.

```
Class.mro()
```

4️⃣8️⃣ What happens when you do `import *`?

Answer

`import *` imports all public names from a module. It:

- Pollutes namespace

- Makes code harder to read
- Can override existing names

Avoid it except in controlled cases.

4️⃣9️⃣ Why is `dict.get()` preferred over direct indexing?

Answer

`dict[key]` raises `KeyError` if key is missing. `dict.get(key)` returns `None` (or default), avoiding exceptions.

Preferred when missing keys are expected.

5️⃣0️⃣ What is the difference between `enumerate()` and `range(len())`?

Answer

`enumerate()` is:

- Cleaner
- Safer
- More Pythonic

```
for i, v in enumerate(items):
    ...
```

Avoids index errors and improves readability.

5️⃣1️⃣ Why should `except Exception` be avoided?

Answer

It catches **too many errors**, including programming mistakes. It can hide bugs and make debugging harder.

Always catch **specific exceptions**.

5️⃣2️⃣ What is the difference between `logging.error()` and `logging.exception()`?

Answer

- `logging.error()` logs a message
- `logging.exception()` logs message **with stack trace**

Use `logging.exception()` inside `except` blocks.

5️⃣3️⃣ What happens if you forget to close a file?

Answer

- File descriptors remain open
- Resource leaks occur
- Can exhaust OS limits

Always use `with open()` to ensure closure.

5️⃣4️⃣ Why is `pickle` faster than JSON?

Answer

`pickle`:

- Is Python-specific
- Preserves object structure
- Uses binary format

JSON is slower but safer and language-agnostic.

5️⃣5️⃣ What is the difference between `asyncio.gather()` and `await`?

Answer

- `await` waits for **one coroutine**
- `asyncio.gather()` runs **multiple coroutines concurrently**

```
await asyncio.gather(task1(), task2())
```

5️⃣6️⃣ Why should blocking code not be used in async functions?

Answer

Blocking code **freezes the event loop**, stopping all other tasks. Use async-compatible libraries or offload to threads/processes.

5️⃣7️⃣ What is a race condition in Python?

Answer

Occurs when multiple threads access shared data simultaneously. Result depends on timing, causing unpredictable behavior.

Use locks or avoid shared state.

5️⃣8️⃣ What is the difference between CPU-bound and I/O-bound tasks?

Answer

- CPU-bound → heavy computation (use multiprocessing)
- I/O-bound → waiting on external systems (use threading/async)

Choosing wrong model hurts performance.

5️⃣9️⃣ Why does Python prefer “explicit over implicit”?

Answer

This is from **The Zen of Python**. Explicit code:

- Is easier to read
- Easier to debug
- Easier to maintain

Python prioritizes clarity over cleverness.

6️⃣0️⃣ What is the biggest mistake mid-level Python developers make?

Answer

- Overusing language features
- Ignoring performance measurement
- Writing clever but unreadable code
- Not understanding concurrency trade-offs

🧠 Python Interview Questions & Answers (Core + OOP + Advanced)

🔹 LISTS

1️⃣ How are Python lists implemented internally?

Answer: Python lists are implemented as **dynamic arrays**. They store references to objects in contiguous memory, allowing O(1) index access. When a list grows, Python allocates extra space to reduce resizing cost.

2️⃣ Why is `append()` faster than `insert()`?

Answer: `append()` adds elements at the end (amortized O(1)). `insert()` shifts elements to make space, which takes O(n). This difference matters in performance-critical code.

3️⃣ When should you avoid using lists?

Answer: Avoid lists when:

- Membership testing is frequent → use `set`
- Keys are needed → use `dict`
- Immutability is required → use `tuple`

🔹 TUPLES

4️⃣ Why are tuples more memory-efficient than lists?

Answer: Tuples have a **fixed size**, so Python does not allocate extra capacity. This reduces memory overhead and improves iteration speed.

5️⃣ Can tuples contain mutable elements?

Answer: Yes. Tuple immutability applies to **structure**, not contents. Mutable elements inside a tuple can still be modified.

🔹 SETS

6️⃣ Why are sets unordered?

Answer: Sets are implemented using **hash tables**. Element position depends on hash values, not insertion order.

7️⃣ Why are sets faster for membership tests?

Answer: Set lookups are **O(1) average time** due to hashing. Lists require O(n) linear search.

🔹 DICTIONARIES

8️⃣ Why must dictionary keys be immutable?

Answer: Dictionary keys must be **hashable**. Mutable objects can change hash values, breaking lookup consistency.

9️⃣ What happens during a dictionary hash collision?

Answer: Python resolves collisions using **open addressing**. It probes alternative slots until a free one is found.

🔹 DEFINING FUNCTIONS

🔹 Parameters & Arguments

🔟 What is the difference between parameters and arguments?

Answer: Parameters are variables in function definition. Arguments are values passed during function call.

1️⃣1️⃣ Why are default arguments evaluated only once?

Answer: Default values are evaluated at **function definition time**, not call time. This improves performance but causes issues with mutable defaults.

1️⃣2️⃣ When should keyword arguments be preferred?

Answer: When:

- Functions have many parameters
- Readability matters
- Order should not matter

They reduce bugs and improve clarity.

1️⃣3️⃣ What is the real use of `*args` and `**kwargs`?

Answer: They enable:

- Flexible APIs
- Forwarding arguments
- Decorators and wrappers

Common in frameworks and libraries.

🔹 RETURN VALUES

1️⃣4️⃣ Can a function return multiple values?

Answer: Python returns a **single tuple**, which can be unpacked. This improves readability without extra objects.

🔹 ANONYMOUS FUNCTIONS

1️⃣5️⃣ When should lambda functions be avoided?

Answer: Avoid lambdas when:

- Logic is complex
- Debugging is required
- Reusability is needed

Use `def` instead.

🔹 `map()`, `filter()`, `reduce()`

1️⃣6️⃣ Why is `reduce()` less commonly used today?

Answer: Because:

- It reduces readability
- Loops and comprehensions are clearer
- `reduce()` hides control flow

Still useful in functional pipelines.

🔹 CLASSES & OBJECTS

1️⃣7️⃣ What happens when you create an object?

Answer: Python:

1. Calls `__new__()` to create object
2. Calls `__init__()` to initialize it

1️⃣8️⃣ What is the difference between instance and class attributes?

Answer:

- Instance attributes → object-specific
- Class attributes → shared across all instances

🔹 ENCAPSULATION

1️⃣9️⃣ How does Python enforce encapsulation?

Answer: Through **convention**, not restriction. Single underscore → protected Double underscore → name mangling

---

🔹 INHERITANCE

2️⃣0️⃣ Why is multiple inheritance risky?

Answer: It can cause:

- Method ambiguity
- Complex MRO
- Maintenance issues

Use composition when possible.

🔹 POLYMORPHISM

2️⃣1️⃣ How does Python achieve polymorphism without interfaces?

Answer: Through **duck typing**. Behavior matters more than type hierarchy.

🔹 ABSTRACTION

2️⃣2️⃣ When should abstract base classes be used?

Answer: Use ABCs when:

- Enforcing contracts
- Designing frameworks
- Preventing incomplete implementations

🔹 METHOD OVERRIDING

2️⃣3️⃣ What rules must overridden methods follow?

Answer: They must:

- Accept compatible arguments
- Preserve expected behavior
- Follow Liskov Substitution Principle

🔹 MRO & `super()`

2️⃣4️⃣ Why is `super()` preferred over direct parent calls?

Answer: `super()` respects MRO and supports cooperative multiple inheritance. Direct calls can break hierarchy.

2️⃣5️⃣ How does Python compute MRO?

Answer: Using **C3 linearization**, ensuring:

- No duplicates
- Consistent order
- Parent precedence

🔹 MAGIC / DUNDER METHODS

2️⃣6️⃣ Why should `__repr__` be unambiguous?

Answer: It's used for debugging and logging. Ideally, it should recreate the object if possible.

2️⃣7️⃣ When is `__call__` useful?

Answer: When objects need to behave like functions. Common in decorators and callable classes.

🔹 DATACLASSES

2️⃣8️⃣ Why prefer dataclasses over regular classes?

Answer: They:

- Reduce boilerplate
- Improve readability
- Auto-generate `__init__`, `__repr__`, `__eq__`

Ideal for data containers.

🔹 ITERATORS

2️⃣9️⃣ What makes an object an iterator?

Answer: It must implement:

- `__iter__()`
- `__next__()`

3️⃣0️⃣ Why do iterators raise `StopIteration`?

Answer: It signals completion of iteration. Used internally by `for` loops.

🔹 GENERATORS

3️⃣1️⃣ How does `yield` differ from `return`?

Answer: `yield` pauses execution and preserves state. `return` terminates function execution.

3️⃣2️⃣ Why are generators memory-efficient?

Answer: They generate values **on demand**, not upfront. Ideal for large or infinite sequences.

🔹 GENERATOR EXPRESSIONS

3️⃣3️⃣ When should generator expressions be avoided?

Answer: When:

- Data must be reused
- Random access is required

Use lists instead.

🔹 DECORATORS

3️⃣4️⃣ Why are decorators evaluated at definition time?

Answer: Because `@decorator` rewrites the function when it is defined, not called. This enables behavior modification before execution.

3️⃣5️⃣ When should class decorators be used?

Answer: For:

- Class-level validation
- Registration (ORMs, plugins)
- Auto-configuration

🧠 More Python Interview Questions & Answers (No Duplicates)

✅ Lists

1) What's the difference between `extend()` and `append()`?

Answer: `append(x)` adds **x as a single element** (even if x is a list). `extend(iterable)` adds **each element from the iterable** to the list. Use `extend()` when you want to merge elements, not nest.

---

2) How do `sort()` and `sorted()` differ in behavior and use?

Answer: `list.sort()` sorts **in-place** and returns `None`. `sorted(iterable)` returns a **new sorted list** without modifying the original. Use `sorted()` when you need immutability or when sorting non-list iterables.

---

3) Why is `pop(0)` slow in a list and what's the alternative?

Answer: `pop(0)` shifts all elements left → **O(n)**. Use `collections.deque` for fast pops from the left → **O(1)**.

✅ Tuples

4) Why are tuples preferred for dictionary keys and set elements?

Answer: Tuples (if they contain only immutable items) are **hashable**, so they can be keys. Lists are mutable and unhashable, so they can't be keys.

---

5) What is tuple unpacking and where is it useful?

Answer: Tuple unpacking assigns multiple variables in one step. Used in returning multiple values, swapping variables, and iterating key-value pairs.

✅ Sets

6) What's the difference between `remove()` and `discard()` in sets?

Answer: `remove(x)` raises `KeyError` if x is missing. `discard(x)` does nothing if x doesn't exist. Prefer `discard()` when absence is acceptable.

---

7) When should you use `frozenset`?

Answer: When you need an **immutable set** that is hashable (usable as a dict key or set element). Useful for caching and fixed configuration sets.

✅ Dictionaries

8) What's the difference between `dict.get()` and `setdefault()`?

Answer: `get()` reads safely with a default, without modifying the dict. `setdefault()` inserts the key with a default if it's missing. Use `setdefault()` for grouping, but prefer `defaultdict` for clarity.

---

9) What is the difference between `.keys()`, `.items()`, `.values()` views?

Answer: They return dynamic views, not copies. If dict changes, views reflect updates. Useful for iteration without extra memory allocation.

---

10) How do you merge two dictionaries safely?

Answer: Use `{**a, **b}` or `a | b` (Python 3.9+). If keys overlap, later dict overwrites earlier keys.

✅ Defining Functions / Arguments

11) What is keyword-only argument and why is it useful?

Answer: Arguments after `*` must be passed by keyword. It improves API clarity and prevents positional mistakes.

```
def f(a, *, debug=False):
    ...
```

---

12) What is positional-only argument and when is it used?

Answer: Arguments before `/` must be positional. Used in APIs to prevent breaking changes and preserve call style.

```
def f(x, /, y):
    ...
```

---

13) What is argument unpacking and when do you use it?

Answer: `*` unpacks iterables into positional args; `**` unpacks dict into keyword args. Used in forwarding calls, wrappers, decorators, and flexible APIs.

---

14) What happens if you return nothing from a function?

Answer: Python returns `None` implicitly. This is useful for procedures (actions) vs functions (computations).

✅ Anonymous Functions + map/filter/reduce

15) Why are list comprehensions often preferred over `map()` and `filter()`?

Answer: They're more readable, debuggable, and Pythonic. Comprehensions also allow inline conditions naturally.

---

16) When is `map()` actually a good choice?

Answer: When applying a function to an iterable without needing extra logic. Especially clean when using built-ins like `str.upper`, `int`, etc.

---

17) What is `reduce()` useful for in real projects?

Answer: Aggregations like combining results, computing cumulative operations, or merging structures. But it can reduce readability—use only when it simplifies logic.

✅ Classes & Objects

18) What are `@property` methods and why use them?

Answer: They allow method logic to be accessed like an attribute. Useful for validation, computed attributes, and maintaining backward-compatible APIs.

---

19) What is the difference between instance methods and class methods?

Answer: Instance methods operate on object state via `self`. Class methods operate on class state via `cls`. Use class methods for alternate constructors and class-wide behavior.

---

20) What's the purpose of `__del__` and why is it discouraged?

Answer: It runs when object is garbage-collected, but timing is unpredictable. Can cause memory/resource issues and circular GC problems—prefer context managers.

✅ Encapsulation / Inheritance / Polymorphism / Abstraction

21) What is "composition over inheritance" and why does it matter?

Answer: Composition builds behavior by using other objects instead of inheriting. It reduces tight coupling and avoids fragile hierarchies.

---

22) How is abstraction achieved in Python without interfaces?

Answer: Using ABCs (`abc` module), Protocols, or duck typing. ABCs enforce contracts; duck typing relies on behavior.

✅ Method Overriding / Multiple Inheritance / MRO / super()

23) What is cooperative multiple inheritance?

Answer: A pattern where all classes use `super()` so each class in the MRO is called exactly once. This avoids duplicate initialization and maintains predictable behavior.

---

24) Why can calling a parent class directly break multiple inheritance?

Answer: Direct calls ignore MRO. This can skip classes or call some twice. `super()` ensures correct traversal.

✅ Magic / Dunder Methods

25) Why implement `__len__` and `__bool__`?

Answer: They control truthiness and size behavior. Objects can define how they behave in `if obj:` checks and `len(obj)` calls.

---

26) What is operator overloading and how is it implemented?

Answer: It means redefining operators for custom objects using dunder methods like:

- `__add__`, `__sub__`, `__eq__`, etc. Used in numeric types, vectors, money classes, etc.

✅ Dataclasses

27) When would you choose dataclasses over namedtuple?

Answer: Use dataclasses when you need:

- Mutability
- Defaults
- Methods
- Type hints and richer behavior Use namedtuple for lightweight immutable records.

---

28) What's the difference between `frozen=True` and normal dataclass?

Answer: `frozen=True` makes instances immutable (no reassignment). It improves safety and allows hashability in certain cases.

✅ Iterators / Generators

29) Why is `iter(callable, sentinel)` useful?

Answer: It creates an iterator that keeps calling a function until sentinel value appears. Useful for reading streams/chunks until EOF without manual loops.

---

30) What is the difference between a generator function and generator expression?

Answer: Generator function uses `def` + `yield`, supports multi-step logic and state. Generator expression is single-expression and compact.

✅ Decorators

31) How do you write a decorator that works with any arguments?

Answer: Use `*args, **kwargs` in wrapper to support any signature. Without this, decorator breaks when decorated function has parameters.

---

32) What are common real-world decorator use cases in production?

Answer:

- Logging and tracing
- Authentication/authorization
- Caching
- Retry and circuit-breakers
- Timing and metrics Used to enforce cross-cutting concerns cleanly.

🧠 Python Interview Question Bank (50 Questions with Answers)

🔹 LISTS

1. Why is list indexing O(1)?

Answer: Lists are dynamic arrays storing references in contiguous memory, allowing direct index access.

---

2. Why is deleting from the middle of a list slow?

Answer: Elements must be shifted to fill the gap, making it O(n).

---

3. When should `deque` be preferred over list?

Answer: When frequent insertions/removals are needed from both ends (O(1)).

---

4. Difference between `copy()` and slicing on a list?

Answer: Both create shallow copies; nested objects remain shared.

---

5. Why does `list.sort()` return `None`?

Answer: To emphasize in-place mutation and avoid confusion with value creation.

🔹 TUPLES

6. Why are tuples hashable but lists are not?

Answer: Tuples are immutable, so their hash never changes.

---

7. Why use single-element tuples with a comma?

Answer: Without a comma, Python treats it as a normal expression.

---

8. Why do tuples iterate faster than lists?

Answer: They have simpler internal structure and no resizing logic.

🔹 SETS

9. Why are sets faster than lists for membership checks?

Answer: Sets use hash tables → O(1) average lookup.

---

10. Why can't sets contain mutable objects?

Answer: Mutable objects are not hashable.

---

11. Difference between `set()` and `{}`?

Answer: `{}` creates a dictionary; `set()` creates an empty set.

🔹 DICTIONARIES

12. Why are dict keys required to be immutable?

Answer: Keys must have a stable hash for consistent lookup.

---

13. How does Python preserve insertion order in dicts?

Answer: From Python 3.7+, insertion order is language-guaranteed.

---

14. Why is `defaultdict` preferred over `setdefault()`?

Answer: Cleaner, faster, and avoids side effects.

---

15. What happens internally during a dict resize?

Answer: Python allocates a larger table and rehashes keys.

🔹 FUNCTIONS

16. Why are functions first-class objects?

Answer: They can be assigned, passed, and returned like any object.

---

17. What is a keyword-only argument?

Answer: Arguments after `*` must be passed by name, improving API safety.

---

18. Why is returning `None` explicit in Python design?

Answer: It clearly differentiates procedures from value-returning functions.

---

19. Why should mutable defaults be avoided?

Answer: They are shared across function calls.

---

20. Why use `*args` and `**kwargs` in decorators?

Answer: To support any function signature safely.

🔹 LAMBDA / FUNCTIONAL TOOLS

21. Why are lambdas limited to one expression?

Answer: To discourage complex logic and improve readability.

---

22. When is `map()` preferred over list comprehension?

Answer: When applying a single function cleanly to all elements.

---

23. Why is `reduce()` controversial?

Answer: It hides control flow and reduces readability.

🔹 CLASSES & OBJECTS

24. What happens when an object is instantiated?

Answer: `__new__()` creates the object, `__init__()` initializes it.

---

25. Difference between class and instance attributes?

Answer: Class attributes are shared; instance attributes are object-specific.

---

26. Why should logic be avoided in `__init__()`?

Answer: Heavy logic makes object creation fragile and slow.

🔹 ENCAPSULATION / INHERITANCE

27. How does Python implement encapsulation?

Answer: Through naming conventions and name mangling.

---

28. Why is inheritance considered fragile?

Answer: Changes in base classes can break subclasses.

---

29. Why prefer composition over inheritance?

Answer: Composition is more flexible and reduces coupling.

---

30. How does Python support polymorphism?

Answer: Through duck typing—behavior matters, not type.

🔹 ABSTRACTION

31. Why use abstract base classes?

Answer: To enforce method contracts and prevent incomplete implementations.

---

32. When should Protocols be preferred over ABCs?

Answer: When structural typing is sufficient and inheritance is undesirable.

🔹 METHOD OVERRIDING / MRO / SUPER

33. What is method overriding?

Answer: Replacing a parent method with child-specific behavior.

---

34. Why is `super()` important in multiple inheritance?

Answer: It respects MRO and avoids duplicate method calls.

---

35. What is C3 linearization?

Answer: Algorithm used by Python to compute consistent MRO.

🔹 MAGIC / DUNDER METHODS

36. Why implement `__eq__()`?

Answer: To define logical equality instead of identity comparison.

---

37. Difference between `__repr__` and `__str__`?

Answer: `__repr__` is developer-focused; `__str__` is user-friendly.

---

38. What is `__call__()` used for?

Answer: Allows objects to behave like functions.

🔹 DATACLASSES

39. Why do dataclasses improve maintainability?

Answer: They reduce boilerplate and improve clarity.

---

40. What does `init=False` do in dataclasses?

Answer: Excludes the field from auto-generated `__init__()`.

🔹 ITERATORS

41. What makes an object iterable?

Answer: Implementing `__iter__()`.

---

42. Difference between iterable and iterator?

Answer: Iterables create iterators; iterators maintain state.

🔹 GENERATORS

43. Why does `yield` preserve state?

Answer: Execution frame is stored between calls.

---

44. Why can generators only be iterated once?

Answer: They maintain internal state and get exhausted.

---

45. When should generators be avoided?

Answer: When data must be reused or accessed randomly.

🔹 DECORATORS

46. What happens at `@decorator` execution time?

Answer: Function is wrapped at definition time, not call time.

---

47. Why use `functools.wraps`?

Answer: To preserve function metadata.

---

48. When are class decorators useful?

Answer: For registration, validation, and configuration logic.

---

49. Why can decorators hurt readability?

Answer: They add abstraction layers and hide control flow.

---

50. What is the biggest decorator misuse?

Answer: Using decorators for business logic instead of cross-cutting concerns.
